



REUTLINGEN UNIVERSITY
FAKULTÄT

INFORMATIK

WiSe 2025 | 01.09.2025 – 28.02.2026

Praktikumsbericht

Pre-Integration Connected Features @ Mercedes-Benz

Martin Lauterbach

Matrikel-Nr.: 810087

martin.lauterbach@student.reutlingen-university.de

Betreuer: **Martin Russold**

martin.russold@mercedes-benz.com

Praktikantenamt: Prof. Dr. Tullius

Monat	Arbeitstage	Stunden	Krank
September 2025	22	158,53	–
Oktober 2025	21	149,26	–
November 2025	20	146,38	–
Dezember 2025	15	164,18	–
Januar 2026	15	143,39	1
Februar 2026	18	144,92	1
Gesamt	111	906,66	2

9 Urlaubstage | 3 Gleittage | 5 Feiertage

Abgabedatum: 24. Februar 2026



Hochschule Reutlingen
Reutlingen University

Inhaltsverzeichnis

1	Einleitung	1
2	Praktikumsziele & -erwartungen	1
3	Wochenberichte	1
4	Unternehmensprofil	9
5	Beschreibung der durchlaufenen Abteilung	10
6	Methodik	11
7	Praktikumsaufgaben & -projekte	11
8	Selbstreflexion & Lernzielanalyse	22
9	Liste der eingesetzten Software-Werkzeuge	26
10	Themenvorschläge für Bachelorarbeiten	27
11	Fazit und Ausblick	28
12	Danksagung	28
	Literaturverzeichnis	29
	Glossar	29
	Erklärung zur wissenschaftlichen Arbeit	30

1 Einleitung

Der vorliegende Bericht dokumentiert mein Praxissemester bei der Mercedes-Benz Group AG im Wintersemester 2025/2026. Das Praktikum wurde in der Abteilung Pre-Integration Telematik & Connected Services am Standort Sindelfingen absolviert und umfasste den Zeitraum von September 2025 bis Februar 2026.

Mich reizte die Möglichkeit, Software nah an der Hardware zu entwickeln – eingebettete, OTA-flashbare Steuergeräte, die untereinander per Automotive Ethernet und CAN sowie über Mobilfunk mit einem verteilten Backend kommunizieren. Das Gesamtsystem ist hochkomplex: kognitive Komplexität, Systemkomplexität, eine Vielzahl spezialisierter Werkzeuge. Diese Herausforderung hat mich motiviert, das Praktikum in dieser Abteilung anzutreten.

Die Abteilung ist Teil des Bereichs Research & Development und verantwortet die Pre-Integration Telematik & Connected Services von vernetzten Fahrzeugdiensten. In diesem Umfeld konnte ich sowohl organisatorische Aufgaben im Fahrzeugpool-Management als auch technische Projekte in der Softwareentwicklung, Datenanalyse und Fahrzeugdiagnose übernehmen.

Ziel dieses Berichts ist es, die durchgeführten Tätigkeiten, eingesetzten Werkzeuge und gewonnenen Erkenntnisse strukturiert darzustellen und kritisch zu reflektieren.

2 Praktikumsziele & -erwartungen

Vor Beginn des Praktikums wurden folgende Lernziele formuliert, die im Verlauf des Praxissemesters erreicht werden sollten:

1. **Einblick in die Arbeitsweise eines großen Automobilherstellers:** Verständnis für die Prozesse, Strukturen und Kommunikationswege innerhalb eines Konzerns mit über 160.000 Mitarbeitenden gewinnen.
2. **Praktische Softwareentwicklung im professionellen Umfeld:** Mindestens ein eigenständiges Software-Tool entwickeln, das im täglichen Testbetrieb des Teams eingesetzt wird – inklusive Tests, Dokumentation und Auslieferung.
3. **Umgang mit großen Datenmengen:** Dashboards in Splunk und Azure Data Explorer erstellen, die dem Team die datenbasierte Bewertung von Softwareständen ermöglichen.
4. **Software auf vernetzter Hardware:** Verständnis für Software auf eingebetteten, OTA-flashbaren Steuergeräten aufbauen – einschließlich deren Vernetzung untereinander (CAN) und mit dem Backend.
5. **Zusammenarbeit im Team:** Erfahrungen in der teamübergreifenden Kooperation, in der Kommunikation mit verschiedenen Stakeholdern und im eigenverantwortlichen Arbeiten sammeln.

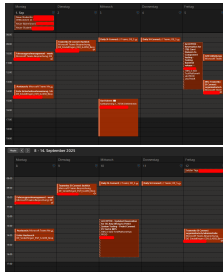
Die Erreichung dieser Ziele wird in Abschnitt 8 (Selbstreflexion & Lernzielanalyse) ausgewertet.

3 Wochenberichte

■ CarlaDLTAnalysis ■ EVDashboard ■ Test Session Tool

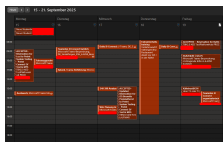
KW	Zeitraum	Schwerpunkte	Tage
<i>September</i>			
36	01.–07.	Onboarding, Sicherheitsunterweisung, Teamvorstellung	5
37	08.–14.	Erster Austausch, CarPool-Einarbeitung	5
38	15.–21.	Splunk/Filter-Engine-Einführung, Fahrsicherheitstraining, erste Git-Commits	5
39	22.–28.	Lokale Filter-Engine-Einrichtung, Normalized Software Evaluation, Erste-Hilfe-Kurs	5
<i>Oktober</i>			
40	29.–05.	Normalized Software Evaluation, Splunk-Dashboards, Radwechsel	5
41	06.–12.	EV-DLT-Filter, ADX-Dashboard, Logger-Adapterkabel, HV-Schulung	5
42	13.–19.	EV-Testing Dashboard (Azure), CarPool-Fahrzeugprüfung, EVRA-Austausch	5
43	20.–26.	CarPool, DLT-Filter, Family@ESH-Vorbereitung, Radwechsel	5
<i>November</i>			
44	27.–02.	Family@ESH, DLT-Filter EV/CarPlay, CarPool-Umfahrten	4
45	03.–09.	DLT-Filter, Radwechsel, Flash-Hell, ADX-Dashboard	5
46	10.–16.	ADX-Dashboard, Splunk Artemis, Radwechsel, Logger-Kabel	5
47	17.–23.	Logger-Adapterkabel (Labor), CarPool, Flash-Hell, Indien-Meeting	5
48	24.–30.	Splunk-Dashboards, Indien-Schulung (RDI-Team), Logger-Kabel, Crashtest	5
<i>Dezember</i>			
49	01.–07.	Logger-Einbau/-Konfiguration, Flash-Hell, Test Session Tool Start	5
50	08.–14.	Normalized Software Splunk, Test Session Tool Entwicklung, Logger-Kabel	5
51	15.–21.	Weihnachtserprobung, Test Session Tool Build, Logger-Ausbau	4
<i>Winterpause: 22. Dezember 2025 – 4. Januar 2026</i>			
<i>Januar</i>			
02	05.–11.	Test Session Tool Debugging/Features, CarPool, ADX-VVR	4
03	12.–18.	Test Session Tool (Auth, Pipeline, Packaging), Indien-Schulung, Sicherheitsunterweisung	5
04	19.–25.	Test Session Tool (Multi-Browser, Tests, Stabilität), RDI-Schulung	5
<i>Februar</i>			
05	26.–01.	Test Session Tool (v0.8–0.9), RDI Normalized Software, Langzeittests	5
06	02.–08.	Test Session Tool (Playwright, UI-Redesign), Testfahrt, Kundencenter	5
07	09.–15.	Test Session Tool (Browser-Auth, Stabilität), Pressefahrt-Abnahme	5
08	16.–22.	Pressefahrt-Abnahme, Praktikumsbericht, Übergabe	5
09	23.–01.	TST v1.1.0 Präsentation, Abschlussgespräch, letzter Arbeitstag	4

Tabelle 1: Übersicht der Praktikumswochen

KW 36–37: 01.–14. September 2025

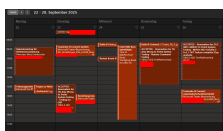
In meiner ersten Woche bei Mercedes-Benz in der Abteilung Pre-Integration Telematik & Connected Services absolvierte ich die Sicherheitsunterweisung und die arbeitsplatzbezogene Einweisung. Gemeinsam mit einer weiteren neuen Studentin und einem neuen Bacheloranden wurde ich im Team willkommen geheißen. In der Woche lernte ich die Teamstruktur und die organisatorischen Abläufe kennen, nahm an der Abteilungsreko teil und erhielt eine erste Einführung in das CarPool-Management durch die Bearbeitung von Fahrzeugreservierungen.

Die zweite Woche stand im Zeichen des weiteren Onboardings. Ich hatte meinen ersten fachlichen Austausch mit meinem Betreuer Martin Russold und begann, mich intensiver mit den CarPool-Prozessen vertraut zu machen, insbesondere der Fahrzeugverwaltung über das interne Buchungssystem und der Bearbeitung von Reservierungsanfragen.

KW 38: 15.–21. September 2025

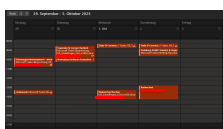
In dieser Woche erhielt ich meine erste technische Einführung in Splunk und die interne Logdaten-Filter-Engine, die das Rückgrat der DLT-Log-Analyse bilden. Parallel dazu nahm ich am Fahrsicherheitstraining teil und besuchte die ShArE-Veranstaltung. Meine ersten Git-Commits entstanden im CarlaDLTAnalysis-Repository, wo ich Mermaid-Diagramme und eine PlantUML-Dokumentation für das schnelle Onboarding neuer Teammitglieder erstellte.

Commits (4): • Mermaid + PlantUML Onboarding-Doku • Dokumentationsordner angelegt
• README: Datenfluss-Doku • PlantUML-Diagramm hochgeladen

KW 39: 22.–28. September 2025

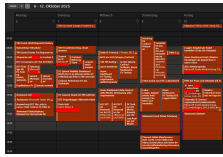
Der Schwerpunkt dieser Woche lag auf der Einrichtung der lokalen Filter-Engine-Umgebung auf meinem Arbeitsrechner sowie der Teilnahme am Review Board für das Projekt Normalized Software Evaluation. Ich absolvierte den ganztägigen Erste-Hilfe-Kurs und arbeitete mich weiter in die DLT-Filterentwicklung ein. Meine ersten eigenen Code-Änderungen am Stability-Filter wurden über GitLab gemergt.

Commits (2): • Merge: Phils Stability-Änderungen • Stability-Filter: Ontime-Extraktion

KW 40: 29. September – 05. Oktober 2025

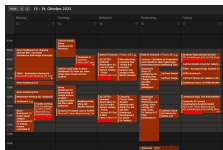
In der fünften Woche arbeitete ich intensiv an der Normalized Software Evaluation: Ich entwickelte Splunk-Dashboards für die normalisierte Stabilitätsbewertung, erstellte Defect-Tickets und organisierte die zugehörigen Aufgaben. Parallel dazu übernahm ich erstmals eigenständig CarPool-Aufgaben wie die Buchung und Durchführung von Radwechseln an Versuchsfahrzeugen.

Commits (7): • Stability civicOnTime+kmticker • Merge stability branch • Filter für 0.7 hinzugefügt • Official SW-Stand choosing • Merge + Dashboard patch • Normalized SW: Dashboard-Logik

KW 41: 06.–12. Oktober 2025

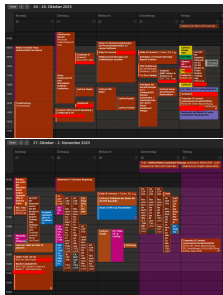
Diese Woche war die bislang vielfältigste: Ich entwickelte DLT-Einträge für EV-Testing in RegEx-Muster und Java-Filter, startete die Erstellung des Azure Data Explorer (ADX) EV-Testing Dashboards und begann im Labor mit dem Bau von Logger-Adapterkabeln durch Mikrolöten. Zusätzlich nahm ich an der HV-Sensibilisierungsschulung teil, pflegte die CarPool-E-mails und besuchte die Splunk Normalized Software Evaluation weiter.

Commits (4): • Fix Sprint-Selection-Filter Bug • Dynamic Aggregation FUP1 • Bar Event Token zu Filterwerten • EE-Klausur left/inner join Logik

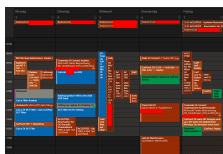
KW 42: 13.–19. Oktober 2025

Der Fokus lag auf der intensiven Weiterentwicklung des EV-Testing Dashboards in Azure: Ich implementierte Ladeprozess-Analysen, Einzelsignal-Analyse-Dashboards und die tabellarische Darstellung aller Charging-Werte. Parallel prüfte ich alle Versuchsfahrzeuge auf vorhandene Tablets und Festplatten und füllte fehlende Komponenten auf. Das Dashboard-Meeting mit Martin Russold und der EVRA-Austausch zur Fehleranalyse rundeten die Woche ab.

Commits (43): • EVICT-Dashboard: Initialisierung • Basis-Konfiguration hochgeladen • 31× Update EVICT Dashboard • 4× Update (Di) • 2× Update (Mi) • 4× Update (Do)

KW 43–44: 20. Oktober – 02. November 2025

Diese beiden Wochen waren stark von CarPool-Aufgaben geprägt: Fahrzeugumfahrten aufgrund von Baustellenarbeiten am ESH, Radwechsel, Flash-Vorbereitungen und die Bearbeitung von Reservierungs-E-mails. Fachlich entwickelte ich DLT-Filter für EV-Testing-Signale und CarPlay in der lokalen Filter-Engine-Umgebung und bereitete das Azure Dashboard zur Präsentation vor. Die Family@ESH-Veranstaltung der Abteilung fand in KW 44 statt, bei der ich als Helfer mitwirkte. Organisatorisch unterstützte ich zudem die Verteilung von Stellenausschreibungen auf Hochschulwebseiten und dokumentierte meine Projektarbeit in Confluence.

KW 45: 03.–09. November 2025

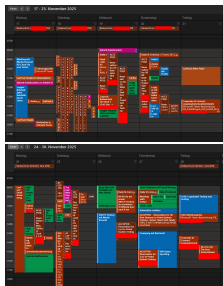
Diese Woche war geprägt von intensiver CarPool-Arbeit mit zahlreichen Radwechseln und der Vorbereitung von Versuchsfahrzeugen. Ich arbeitete weiter an den DLT-Filtern für EV-Testing und CarPlay in der Filter-Engine, erweiterte das ADX-Dashboard um VVR-Ignition-Daten und richtete meinen Laptop für das Flashen von Steuergeräten ein (Flash-Hell). Die Dokumentation der Flash-Abläufe erstellte ich für nachfolgende Praktikanten.

Commits (6): • 3× Update EVICT Dashboard • README: ADX-Zugangslinks • Mermaid: DLT→Splunk Kette • Update EVICT Dashboard

KW 46: 10.–16. November 2025

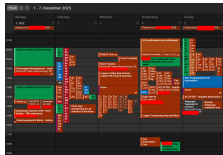
Der Schwerpunkt lag auf der parallelen Weiterentwicklung von Azure- und Splunk-Dashboards: Ich fügte dem ADX-Dashboard Backend-Zeiten, VVR-Log-Zeiten und die VVR-Delay-Analyse nach TCU-eStand hinzu und brachte die Splunk Artemis-Dashboards auf den neuesten Stand. Im Labor lötete ich weitere X2E-Kanalerweiterungs-Kabel. Die zahlreichen Radwechsel an Versuchsfahrzeugen setzten sich fort.

Commits (4): • Update EVICT Dashboard • README: Filter-Dokumentation • Merge: Doku-Branch integriert • Update EVICT Dashboard

KW 47–48: 17.–30. November 2025

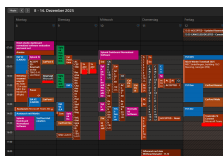
Diese zwei Wochen standen im Zeichen der Logger-Adapterkabel und des Beginns der RDI-Schulung. Im Labor stellte ich weitere Kabelsätze her und testete einen fertigen Satz erstmals im Versuchsfahrzeug. Das erste Meeting mit dem indischen RDI-Kollegenteam zum Thema CarPlay-Datenanalyse markierte den Beginn der Splunk-Schulungsinitiative. In KW 48 folgten die Fertigstellung der Splunk-Dashboards und die erste Schulungssession. Die dritte Flash-Hell-Session fand statt, und ich nahm an der Field Data Community sowie einem Froncrash als Beobachter teil. CarPool-Aufgaben wie Radwechsel und Fahrzeugtransfers nahmen weiterhin viel Zeit in Anspruch.

Commits (3): • Revert + CarPlay-Filter-Features • Update EVICT Dashboard

KW 49: 01.–07. Dezember 2025

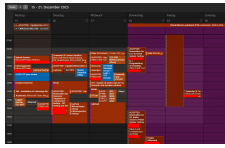
In meiner ersten Dezemberwoche lag der Schwerpunkt auf dem Einbau und der Konfiguration von Logger-Adapterkabeln in den Versuchsfahrzeugen der Flotte, inklusive Firmware-Updates der Logger und Tablet-Apps. Die vierte Flash-Hell-Session fand statt. Parallel begann ich mit der Dokumentation und den ersten Architekturdiagrammen für das Test Session Tool. Ich analysierte empirisch die Refresh-Zeiten zwischen der Fahrzeugdatenbank, dem Fleet Builder und dem Buchungssystem.

Commits (4): • System arch + flow diagrams • Studio Splunk Normalized SW Eval • Timestamped folder logging • Updated README + requirements

KW 50: 08.–14. Dezember 2025

Der Fokus verschob sich zunehmend auf die Softwareentwicklung: Ich finalisierte das Splunk Studio Dashboard für die Normalized Software Evaluation mit umfangreichen Layout- und Filterfunktionen. Gleichzeitig begann die intensive Entwicklung des Test Session Tools – ich implementierte das Config-Panel, den Channel Monitor mit Audio-Benachrichtigungen, die skalierbare Authentifizierung und die modulare Architektur. Im Labor stellte ich die zweite Generation der Logger-Adapterkabel (EIA V2) fertig.

Commits (44): • Normalized SW: Farben, Layout • README: How-to-dev • Normalized SW: Sprint-Kurzform • Config + ControlPanel erstellt • Normalized SW: FUP-übergreifend • Channel Monitor + Audio • Scaleable Authentication • Modulare Ordnerstruktur • Build-Pipeline + Tests ... +35 weitere

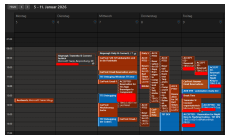
KW 51: 15.–21. Dezember 2025

Die letzte Woche vor der Winterpause war intensiv: Ich bereitete die Versuchsfahrzeuge für die Weihnachtserprobung vor, baute Logger aus verschrottungsreifen Fahrzeugen aus und installierte Logger-Adapterkabel in den Erprobungsfahrzeugen. Beim Test Session Tool erstellte ich den ersten Standalone-Build als .exe, implementierte den zirkulären Videopuffer und begann mit der KI-gestützten Defektbeschreibung (DefectAI). Das Tool wurde erstmals Kollegen vorgeführt.

Commits (20): • Peer-Review Merge • Zirkulärer Videopuffer • Trace Cutter + Logging
• Standalone .exe + Logo + KI • PyInstaller --windowed • Merge ffmpeg video rework
...+14 weitere

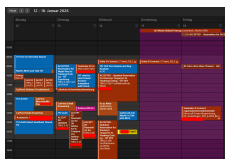
Winterpause: 22. Dezember 2025 – 4. Januar 2026

Betriebsruhe bei Mercedes-Benz. Keine Arbeitstätigkeit.

KW 02: 05.–11. Januar 2026

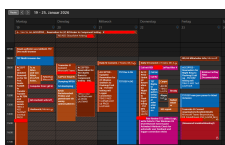
Nach der Winterpause stieg ich direkt in die Weiterentwicklung des Test Session Tools ein: Ich debuggte Codec-Probleme und den Windows-TTS-Fehler, implementierte das Piper-Voice-Modell für Offline-Sprachbenachrichtigungen und restrukturierte die gesamte Applikation mit verbesserter Dokumentation, KI-Integration und Thread-Management. Parallel nahm ich die CarPool-Aufgaben wieder auf und begann mit der Entwicklung der ADX-VVR-Automatisierung.

Commits (17): • Piper Voice + Offline-TTS • Restructured + KI + Monitor • Global Audio Lock • Windows-Kompatibilität • ADX VVR Extraction + Docs • Browser Lock bei VAM-Report ...+11 weitere

KW 03: 12.–18. Januar 2026

Diese Woche widmete ich mich intensiv der Stabilisierung des Test Session Tools: Ich implementierte die eventbasierte Browser-Authentifizierung, das PackageManager-System zur organisierten Artefaktverwaltung und verbesserte die Start- und Shutdown-Erfahrung grundlegend. Die Schulung mit dem RDI-Kollegen zur DLT-Analyse für CarPlay wurde fortgesetzt. Ich absolvierte die jährliche Sicherheitsunterweisung für Erprobungsfahrten.

Commits (12): • Auth Heartbeat + Timeout-Fix • Refactor Auth + Result Handling • PackageManager für Artefakte • Startup/Shutdown verbessert • Auth-Flow: Paralleler Check ...+7 weitere

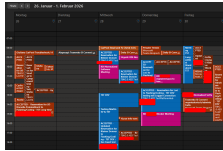
KW 04: 19.–25. Januar 2026

In der vierten Januarwoche arbeitete ich an der Multi-Browser-Unterstützung für das Test Session Tool, implementierte Split-Screen-Anzeigen für VAM-, ADX- und API-Browser und führte zahlreiche Tests im Fahrzeug durch. Ich löste komplexe Probleme wie den Windows COM-Threading-Crash und Audio-Verzerrungen bei der Aufnahme. Das Schulungsmeeting mit dem RDI-Kollegen zum Thema Normalized Software wurde fortgeführt.

Commits (70): • CLAUDE.md für CarlaDLTAnalysis • Split-Screen VAM/ADX/API • Fix Windows COM Crash • Fix Audio Recording 36% • Dynamische Pipeline-Vis. • src/

Restrukturierung komplett • VAM JSON Parser + Tests • Parallel Processing Config
• Video Registry Deduplizierung ...+61 weitere

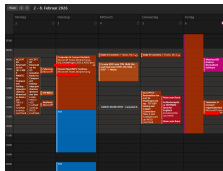
KW 05: 26. Januar – 01. Februar 2026



Der Höhepunkt der Test Session Tool-Entwicklung: Ich implementierte die High-Speed-Videoaufnahme mit ThreadedFrameGrabber, das Queue-Backpressure-System, die KI-Konfidenzwerte für DefectAI, die Video-Registry für deduplizierte Aufnahmen und zahlreiche UX-Verbesserungen. Das Tool erreichte Version 0.9. Parallel fanden ein Langzeit-Standalone-Test und das RDI-Meeting zur Normalized Software Evaluation statt.

Commits (73): • Enter MainLoop Early Pattern • Loading Screen + Config Fixes
• ThreadedFrameGrabber • Queue Backpressure • DefectAI Konfidenzwerte • Fix Video Timing
• Version 0.8.5 • VAM Parallel Watcher Arch. • Version 0.9 finalisiert ...+64 weitere

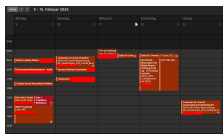
KW 06: 02.–08. Februar 2026



In dieser Woche begann ich mit der Umstellung des Test Session Tools auf Playwright als Browser-Automatisierungsschicht und redesignte die gesamte Dashboard-Oberfläche mit responsivem Layout-System und einheitlichem Farbthema. Ich nahm an einer gemeinsamen Navi/MFS-Testfahrt teil und besuchte das Mercedes-Benz Kundencenter in Sindelfingen. Das Meeting mit dem RDI-Kollegen zur Normalized Software wurde fortgesetzt.

Commits (82): • Playwright Browser-Automation • Responsive Layout + Dashboard • UI Theme Light Palette • Logger Auto-Reconnection • Dashboard nach Spezifikation • Build-Skript + Datenpakete • Preflight Check UI • Config Anti-Monolith Refactor • Auth-Erkennung verbessert ...+73 weitere

KW 07: 09.–15. Februar 2026



In meiner vorletzten aktiven Woche lag der Schwerpunkt auf der Fertigstellung der Browser-Authentifizierungsarchitektur des Test Session Tools mit dem 4-Phasen-Startup-Login-Prozess, Lazy-Browser-Controllern und verbesserter Crash-Recovery. Ich unterstützte die Pre-Int Telematik-Abnahme für die Pressefahrzeuge und führte das CarPlay-Board der Normalized Software Evaluation für das RDI-Team zu Ende.

Commits (50): • 4-Phasen Startup-Auth • LazyBrowserController • Paralleler Cleanup + Timeout • AG-Grid Extraktion + CSV • 4-Phasen Login Refactor • Browser Crash Recovery • TTS Fix: Piper 1.4.1 • Session Recorder 5fps • TurboJPEG: 90% RAM-Reduz. ...+41 weitere

KW 08–09: 16. Februar – 01. März 2026

In der letzten vollen Arbeitswoche unterstützte ich die Pre-Int Telematik-Abnahmen der Pressefahrzeuge und nahm an der Sicherheitsunterweisung für Steer-by-Wire teil. Parallel widmete ich mich der Fertigstellung des Praktikumsberichts und der Übergabedokumentation für nachfolgende Praktikanten.

Meine letzte Woche im Praktikum: Am Montag hatte ich Urlaub. Am Dienstag präsentierte ich Version v1.1.0-wabi-sabi von TST dem Team – dafür hatte ich ein Video aufgenommen, in dem die TTS-Funktion des Tools selbst die Narration übernahm, und dieses anschließend mit DaVinci Resolve geschnitten. Die Präsentation stieß auf große Begeisterung: Das Team applaudierte und zeigte sich beeindruckt vom Funktionsumfang und der Innovation des Tools. Am Donnerstag fand das Abschlussgespräch mit dem Teamleiter, dem stellvertretenden Teamleiter und meinem Betreuer Martin Russold statt, in dem wir das Praktikum reflektierten. Am Freitag, dem 27. Februar, war mein letzter Arbeitstag bei Mercedes-Benz.

Commits (52): • Welcome-Tab + Info-Panels • Edge-TTS Migration • Schema-Hex-Validierung • QuickStart-Guide • Video-Automator Split • UML + Callgraph-Gen. • Doku-Bereinigung • Marker-ID-Deduplizierung • Netzwerk-Paketmitschnitt ...+43
weitere

4 Unternehmensprofil

4.1 Geschichtliche Entwicklung

Mercedes-Benz geht auf Karl Benz und Gottlieb Daimler zurück, die 1886 unabhängig voneinander die ersten Automobile entwickelten. 1926 fusionierten ihre Unternehmen zur Daimler-Benz AG. Nach der Fusion mit Chrysler (1998–2007) und der Umbenennung in Daimler AG wurde das Unternehmen Anfang 2022 in die Mercedes-Benz Group AG umfirmiert; das Nutzfahrzeuggeschäft wurde als Daimler Truck abgespalten Mercedes-Benz Group AG, 2024a.

4.2 Geschäftsfelder und Kennzahlen

Die Mercedes-Benz Group AG (rund 166.000 Mitarbeitende, ca. 146 Mrd. € Umsatz 2024) gliedert sich in drei Bereiche: **Mercedes-Benz Cars** (Pkw von der Kompakt- bis zur Oberklasse inkl. Elektrofahrzeuge wie EQS und EQE), **Mercedes-Benz Vans** (Sprinter, Vito, V-Klasse) und **Mercedes-Benz Mobility** (Finanzierung, Leasing, Versicherung). Der Hauptsitz liegt in Stuttgart; der Standort Sindelfingen, an dem dieses Praktikum stattfand, ist eines der größten Forschungs- und Entwicklungszentren des Konzerns Mercedes-Benz Group AG, 2024a.

4.3 Transformation zur softwaredefinierten Mobilität

Mercedes-Benz befindet sich in einer tiefgreifenden Transformation, die den direkten Kontext dieses Praktikums bildet. Die strategischen Schwerpunkte „Lead in electric drive“ und „Lead in car software“ Mercedes-Benz Group AG, 2024c verschieben den Fokus von der reinen Fahrzeugmechanik hin zum softwaredefinierten Fahrzeug. Zentral ist dabei das hausinterne Mercedes-Benz Operating System (MB.OS), das als einheitliche Softwareplattform für alle künftigen Fahrzeuggenerationen dient Mercedes-Benz Group AG, 2024b. MB.OS ermöglicht OTA-Updates, über die neue Funktionen und Fehlerbehebungen direkt an Kundenfahrzeuge ausgeliefert werden – ohne Werkstattbesuch.

Für die Abteilung Pre-Integration Telematik & Connected Services, in der dieses Praktikum stattfand, bedeutet diese Transformation einen grundlegenden Wandel der Absicherungsarbeit: Statt einzelner Hardware-Funktionen müssen nun komplexe, vernetzte Softwaresysteme integriert und validiert werden. Die Qualität der ausgelieferten Software wird durch Big-Data-Analysen – wie die in diesem Praktikum entwickelten Splunk- und ADX-Dashboards – kontinuierlich überwacht. Diese datengetriebene Absicherung ist eine direkte Konsequenz der Tatsache, dass ein Softwarefehler in einem vernetzten Fahrzeug nicht mehr nur das einzelne Fahrzeug betrifft, sondern potenziell die gesamte Flotte.

4.4 Organisatorischer Aufbau

Die Mercedes-Benz Group AG gliedert sich auf Konzernebene in die Geschäftsfelder Cars, Vans und Mobility. Innerhalb von Mercedes-Benz Cars ist der Bereich Research & Development (RD) für die Fahrzeugentwicklung verantwortlich. Die Abteilung Pre-Integration Telematik & Connected Services betreut die Absicherung vernetzter Fahrzeugfunktionen; das Team Pre-Integration Connected Features, in dem dieses Praktikum stattfand, ist dort angesiedelt.

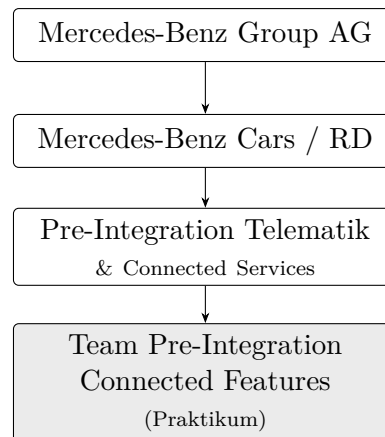


Abbildung 1: Organisatorische Einordnung des Praktikumsbereichs

5 Beschreibung der durchlaufenen Abteilung

Das Praktikum wurde in der Abteilung **Pre-Integration Telematik & Connected Services** absolviert, die dem Bereich Research & Development (RD) am Standort Sindelfingen zugeordnet ist. Der Arbeitsplatz befand sich im Gebäude 84/0 ESH.

5.1 Einordnung im Unternehmen

Der Bereich Research & Development in Sindelfingen ist für die Forschung und Entwicklung der Mercedes-Benz Cars verantwortlich. Die Abteilung Pre-Integration Telematik & Connected Services ist innerhalb dieses Bereichs für die **Absicherung vernetzter Fahrzeugdienste** zuständig: Sie bildet die Schnittstelle zwischen der Softwareentwicklung einzelner Steuergeräte und dem Gesamtfahrzeug. Neue Entwicklungssoftwarestände werden hier integriert und abgesichert, bevor sie in die Serienproduktion oder als Over-the-Air-Update (OTA) an Kundenfahrzeuge ausgeliefert werden.

Im Fokus der täglichen Arbeit stehen die vernetzten Dienste des Fahrzeugs (*Connected Car*). Die Telematik-Steuereinheit (TCU) ist dabei das zentrale Steuergerät: Sie kann Befehle und Daten senden und empfangen – auch an andere Steuergeräte im Fahrzeug. Die HU stellt die Nutzerfunktionen bereit, die TCU sorgt für Kompatibilität und Vielseitigkeit der Anbindung. Über diese Architektur laufen Mercedes me App, Remote-Funktionen, Diagnosedienste und Softwareaktualisierungen.

5.2 Aufgabenspektrum der Abteilung

Die tägliche Arbeit der Abteilung umfasst unter anderem:

- **Softwareintegration und -absicherung:** Neue Softwarelieferungen für die TCU und die HU werden in Sprints getestet und bewertet. Dabei wird gleichzeitig an mehreren Fahrzeuggenerationen gearbeitet – sowohl an Modellen, die sich bereits auf dem Markt befinden, als auch an Fahrzeugen in der Entwicklungsphase.
- **Datenanalyse und Monitoring:** Über Diagnostic Log and Trace (DLT) Protokolle werden Fahrzeugdaten erfasst und mit Big-Data-Analyseplattformen ausgewertet. Dashboards ermöglichen die Überwachung von Softwarestabilität, Connectivity-Kennzahlen und Fehlerhäufigkeiten.
- **Fahrzeugflotten-Management:** Für die Tests steht ein Pool an Versuchsfahrzeugen zur Verfügung, der organisatorisch verwaltet werden muss – von der Buchung über die Diagnose bis hin zur Rückgabe.
- **Defekt-Management:** Erkannte Fehler werden dokumentiert, reproduziert und an die zuständigen Entwicklungsteams weitergeleitet. Hierzu wird ein zentrales Defect-Management-System eingesetzt.

5.3 Das Team

Innerhalb der Abteilung wurde das Praktikum im Team *Pre-Integration Connected Features* absolviert. Das Team umfasst 19 Personen – Festangestellte, Werkstudenten und Praktikanten. Der direkte Betreuer war Martin Russold. Die Zusammenarbeit im Team war kollegial und auf Augenhöhe – als Praktikant wurde ich von Anfang an als vollwertiges Teammitglied behandelt und in alle relevanten Prozesse eingebunden. Die Arbeitsweise ist sprintbasiert, mit regelmäßigen Abstimmungsmeetings und einem Fokus auf eigenverantwortliches Arbeiten.

6 Methodik

Die Arbeitsweise der Abteilung ist sprintbasiert organisiert. In geregelten Zyklen von ungefähr zwei Wochen werden neue Softwarestände für die Telematik-Steuereinheit und die Head Unit getestet und bewertet. Dabei wird zwischen verschiedenen Fahrzeuggenerationen und Plattformen unterschieden, da mehrere Modellreihen parallel betreut werden. Sprint-Ziele werden in einem wöchentlichen Teammeeting abgestimmt; Testergebnisse und erkannte Defekte fließen in Sprint-Reviews ein, die als Entscheidungsgrundlage für die Softwarefreigabe dienen. Erkannte Defekte werden über das zentrale Defect-Management-System dokumentiert, mit Reproduktionsschritten und DLT-Traces versehen und an die zuständigen Entwicklungsteams weitergeleitet.

Für die technischen Projekte in diesem Praktikum wurde ein iterativer Entwicklungsansatz verfolgt: Anforderungen wurden gemeinsam mit dem Betreuer und den Teamkollegen besprochen, in kleinere Arbeitspakete unterteilt und in kurzen Zyklen umgesetzt. Beim Test Session Tool bedeutete das konkret, neue Features zunächst im Simulationsmodus zu entwickeln, dann im Fahrzeug zu validieren und anschließend auf Basis des Nutzerfeedbacks anzupassen. Durch regelmäßiges Feedback – sowohl von Endanwendern im Team als auch vom Betreuer – konnten Ergebnisse frühzeitig validiert und angepasst werden.

Versionskontrolle über Git war kein Bestandteil der täglichen Teamarbeit – das Team arbeitet primär mit dem zentralen Defect-Management-System und Jira zum Flashen und Defekt-Tracking. Git kam isoliert für meine eigenen Projekte zum Einsatz (Azure DevOps). Dokumentation und Aufgabenverwaltung liefen über Confluence und Jira. Für die Datenanalyse kamen Splunk und die Cloud-basierte Datenanalyseplattform ADX als zentrale Werkzeuge zum Einsatz.

7 Praktikumsaufgaben & -projekte

Die Aufgaben während des Praktikums lassen sich in zwei Hauptbereiche unterteilen: Etwa 40% der Arbeitszeit entfielen auf das organisatorische CarPool-Management, die restlichen 60% auf technische Projekte im Bereich Testing sowie Tool- und Dashboard-Entwicklung.

Diese Aufteilung spiegelt die Vielseitigkeit der Abteilung wider: Das CarPool-Management vermittelte organisatorisches Geschick, Stakeholder-Kommunikation und ein praktisches Verständnis der Fahrzeugflotte, das direkt in die technischen Projekte einfluss. Die technischen Projekte wiederum – vom Test Session Tool (174 Python-Quelldateien) über die DLT-Analyse-Pipeline und Big-Data-Dashboards bis hin zur Hardwarearbeit an Logger-Adapterkabeln – boten die Möglichkeit, Softwareentwicklung, Datenanalyse und Fahrzeugdiagnose in der Praxis anzuwenden und zu vertiefen.

Die folgenden Unterkapitel beschreiben die einzelnen Aufgabenbereiche im Detail.

7.1 40% der Zeit: CarPool-Management

Hintergrund Der Abteilung steht ein Pool aus rund 20 Versuchsfahrzeugen zur Verfügung – von Serienmodellen bis zu Vorserienfahrzeugen –, die parallel mit unterschiedlichen Softwareständen betrieben werden. Die Pool-Organisation ist Voraussetzung für den reibungslosen Testablauf, da Verzögerungen durch fehlende Fahrzeuge den Sprint-Zeitplan beeinflussen.

Aufgabenstellung Die Verwaltung erfolgte über das interne Fahrzeugbuchungssystem zur Reservierung, Zeitraumverwaltung und abteilungsübergreifenden Abstimmung. Zum Tagesgeschäft gehörten:

- **Buchung und Koordination:** Reservierungsanfragen bearbeiten, abteilungsübergreifend abstimmen, nach Testdringlichkeit priorisieren
- **Fahrzeugprüfung:** Fahrzeugzustand prüfen – Ladezustand, Reifendruck, Hardware (Tablets, Logger), Softwarestand
- **Wartungskoordination:** Reifenwechsel, Werkstattaufenthalte und Fahrzeugumfahrten bei Baustellenarbeiten organisieren
- **Flash-Vorbereitung:** Fahrzeuge für Flash-Sessions (Steuergeräte-Updates) vorbereiten und Einsatzbereitschaft sicherstellen
- **Sonderereignisse:** Fahrzeuge für Pressefahrt-Abnahmen, Erprobungsfahrten und Events vorbereiten

Ergebnisse Obwohl keine klassische Softwareaufgabe, war das CarPool-Management lehrreich:

- **Stakeholder-Kommunikation:** Tägliche Abstimmung mit Testingenieuren, Werkstatt und Teamleitern erforderte zielgruppengerechte Kommunikation – eine Fähigkeit, die ich auch in technischen Projekten einsetzte.
- **Priorisierung unter Zeitdruck:** Bei begrenzter Fahrzeugverfügbarkeit und parallelen Testanforderungen mussten Entscheidungen schnell und eigenständig getroffen werden.
- **Fahrzeugtechnisches Verständnis:** Der regelmäßige Umgang mit Versuchsfahrzeugen – Softwarestände prüfen, Logger ein-/ausbauen, Steuergeräte flashen – vermittelte praktisches Verständnis, das direkt in meine Dashboards und das Test Session Tool einfluss.
- **Prozessverständnis:** Die Arbeit im CarPool zeigte mir, wie eng organisatorische und technische Prozesse in der Fahrzeugabsicherung verzahnt sind – ein Test kann nur so gut sein wie die Vorbereitung des Fahrzeugs.

7.2 60% der Zeit: Testing sowie Tool- & Dashboard-Entwicklung

Der technische Anteil des Praktikums umfasste mehrere Projekte, die alle im Kontext der Absicherung und Analyse vernetzter Fahrzeugdienste stehen. Das Spektrum reichte von der Entwicklung eines eigenständigen Automatisierungstools über die Erstellung von Big-Data-Dashboards bis hin zur Hardwarearbeit an Logger-Adapterkabeln. Trotz der Vielfalt teilen die Projekte ein gemeinsames Ziel: die Effizienz und Datenqualität der Fahrzeugabsicherung zu verbessern – sei es durch Automatisierung manueller Prozesse (Test Session Tool), durch bessere Sichtbarkeit auf Fahrzeugdaten (Dashboards) oder durch die Bereitstellung der dafür nötigen Hardware-Infrastruktur (Logger-Adapterkabel).

In Summe umfasst der technische Output 174 Python-Quelldateien mit über 1000 Tests (Test Session Tool), ein ADX-Dashboard mit 145 Kusto-Abfragen und 220 Visualisierungen, über 47 Splunk-Dashboard-Konfigurationen, mehrere Java-DLT-Filterklassen sowie zwei Generationen von Logger-Adapterkabeln. Die folgenden Unterkapitel beschreiben die einzelnen Projekte im Detail, jeweils gegliedert in Hintergrund, Aufgabenstellung und Ergebnisse. Querverbindungen zwischen den Projekten werden an den relevanten Stellen hervorgehoben.

7.2.1 Test Session Tool

Hintergrund Testingenieure können während einer Session zahlreiche Daten manuell erfassen: Videoaufnahmen, Diagnose-Traces, VAM-Berichte, Ladedaten aus ADX und Fehlerbeschreibungen. Dieser Prozess ist zeitaufwändig, fehleranfällig und unterbricht den Testfluss.

Aufgabenstellung Ziel war ein vollautomatisiertes Tool, das beim Setzen eines Markers am Logger sämtliche Daten parallel erfasst, analysiert und zu einem Datenpaket bündelt – ohne manuelle Eingriffe, robust für den täglichen Einsatz.

Architektur Das Tool basiert auf einer **Producer-Consumer-Pipeline** Gamma et al., 1994 (Abbildung 2): Der Logger erkennt Marker-Events und verteilt sie über MonitorMarkers in fünf unabhängige Queues. Jeder Consumer verarbeitet seinen Datentyp mit eigenem Timeout – langsame Trace-Extraktion (bis 5 Min) blockiert nicht die Videoextraktion (3 s). Ein ResultConsumer aggregiert alles zu `ticket.json`.

- **VideoConsumer:** Zirkulärer Videopuffer (OpenCV) mit zeitstempelbasierter Extraktion – erfasst Frames vor und nach dem Marker
- **VamConsumer:** Browser-Automatisierung (wxPython WebView2) mit einer Page Freeze-Technik, die Angular.js-Interferenzen umgeht. Playwright übernimmt die Authentifizierung: Tokens werden automatisch beschafft, gehalten und wiederverwendet
- **TraceConsumer:** Ansteuerung des Logger-Subsystems zur Extraktion von DLT-Traces, anschließend automatische Analyse über die eingebettete Civici-Bibliothek
- **AdxConsumer:** Extraktion von Ladedaten aus ADX-Dashboards über drei Fallback-Strategien (AG-Grid API, Netzwerk-Intercept, Scroll-basiert)
- **DefectConsumer:** KI-gestützte Defektbeschreibung – Sprachaufnahme und Videokompromierung werden an die Gemini-API gesendet, die eine strukturierte JIRA-Ticketbeschreibung generiert (optionaler Consumer, gestrichelte Verbindung)

Ein **ThreadManager** überwacht alle Threads per Heartbeat und startet abgestürzte Consumer neu. Die wxPython-GUI bietet Echtzeit-Dashboards, Konfiguration und Statusanzeigen.

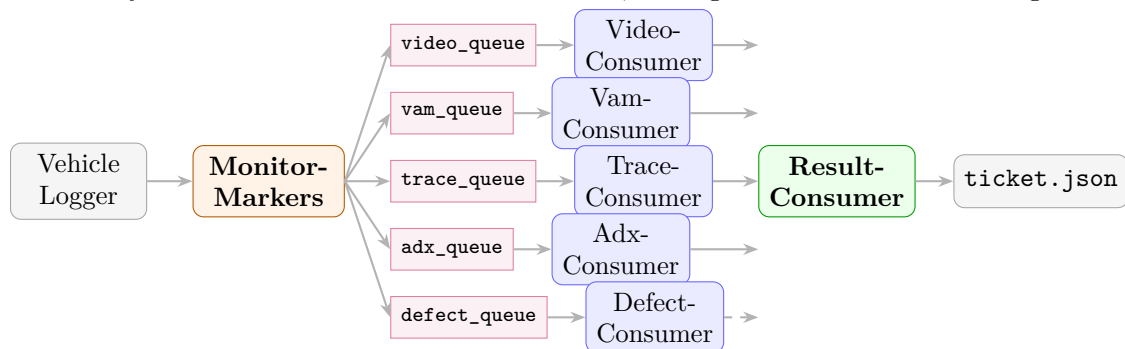
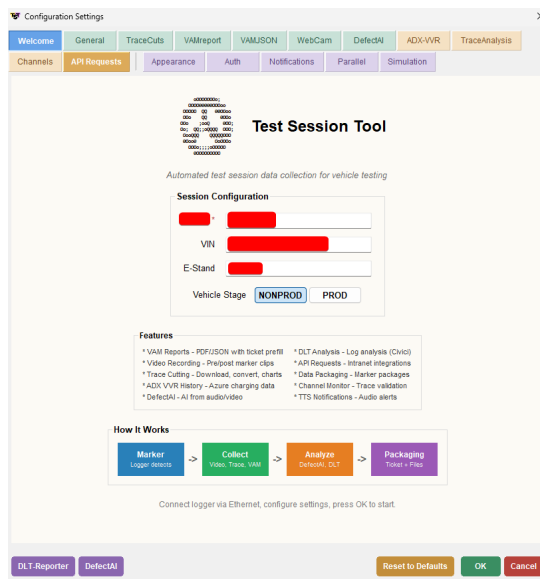
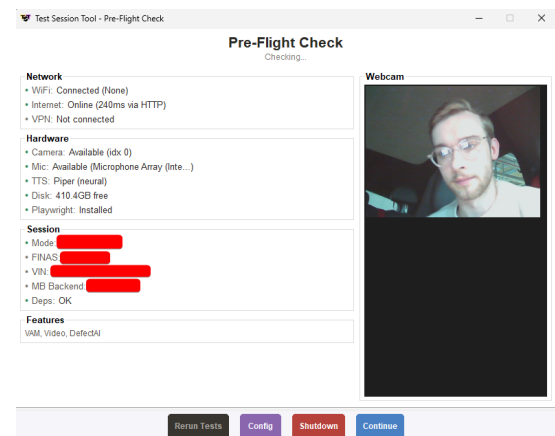


Abbildung 2: Producer-Consumer-Architektur des Test Session Tools (nach README)

Benutzeroberfläche Die GUI hat drei Ansichten: Der **Konfigurationsdialog** (Abb. 3a) erlaubt die Einstellung aller Parameter – FINAS, VIN, Softwarestand, Features – über gruppierte Tabs. Der **Pre-Flight Check** (Abb. 3b) zeigt Netzwerk, Hardware, Konfiguration und Features zur expliziten Bestätigung. Das **Haupt-Dashboard** (Abb. 4a) zeigt Echtzeit-Videofeed, Consumer-Status, Logger-Verbindung und Marker-Historie; ein Debug-Modus ergänzt Thread-Details und Fehlerlogs. Als visuelles Detail rendert der Startbildschirm einen animierten Mercedes-Stern: Ein kompakter 3D-Renderer projiziert den rotierenden Stern mit einer Lichtquelle und stellt jedes Frame als ASCII-Grafik dar.

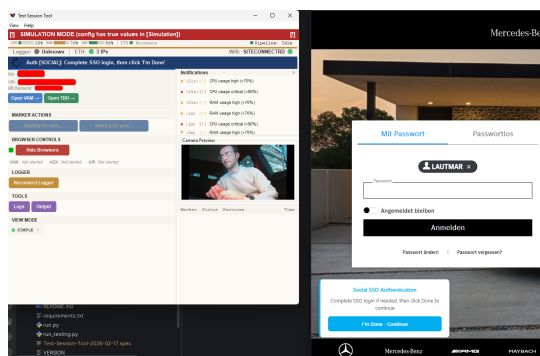


Konfigurationsdialog mit Session-Parametern und Feature-Übersicht

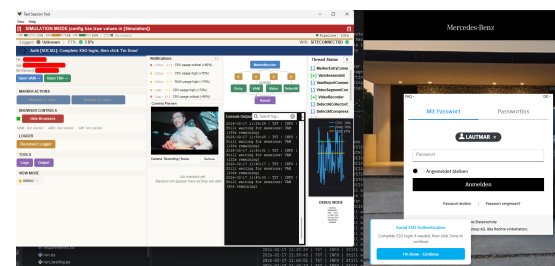


Pre-Flight Check – Systemprüfung vor Session-Start

Abbildung 3: Test Session Tool: Konfiguration und Pre-Flight Check



Dashboard im Standardmodus



Dashboard im Debug-Modus mit Thread-Details

Abbildung 4: Test Session Tool: Haupt-Dashboard in beiden Ansichtsmodi

Ergebnisse Das Tool umfasst 174 Python-Quelldateien, über 1000 Pytest-Tests und eine vollständige Dokumentation. Es reduziert den Zeitaufwand pro Defekt von ca. 20–30 Minuten manueller Datenerfassung auf etwa 4 Minuten bei minimalem Arbeitsaufwand – die freigewordene kognitive Kapazität fließt in das eigentliche Testen. Ein Simulationsmodus mit 12 Feature-Flags ermöglicht Entwicklung ohne Fahrzeughardware.

Verwaltet über semantische Versionierung (v1.0.1-wabisabi); Standalone-.exe-Builds via PyInstaller als Artefakte verteilt. Damit das Wissen und der Geist der Arbeitserleichterung und Effizienz über mein Praktikum hinaus erhalten bleiben, entstand eine umfangreiche Dokumentation (Abbildung 5):

```
docs/
├── architecture/ (9) Pipeline, Threading, Resilience, Startup, Browser-Pool
├── browser/ (9) Auth-Flow, Auth-Detection, PageFreeze, Tiling
├── features/ (8) DefectAI, TraceCutter, VAM, Video, ADX-VVR
├── ItJustIsSo/ (11) HowDoesAMarkerTravel, WhyOneConsumerPerQueue
├── infrastructure/ (3) FileWatcher, FutureCallback, SessionReady
├── ui/ (3) Dashboard-UX, UI-Architektur, Button-Feedback
├── gen_diagrams/ (13) UML-Klassen- und Call-Graphen (SVG)
├── mermaid/ (17) Auth-Flow, Context-Lifecycle, Locking
├── plans/ Referenzdaten: HTML-Quellen, JSON-Dumps, Swagger
├── QuickStart15Minutes.md
└── TST-butItsAStory.md
```

Abbildung 5: Dokumentationsstruktur des Test Session Tools – Wissenserhalt für die Weiterentwicklung

Ein besonderes Dokument ist `TST-butItsAStory.md`: eine narrative Architekturführung, die das gesamte System als Fabrikmetapher erzählt. Jede Figur entspricht einer realen Klasse – der Vorarbeiter ist `MonitorMarkers`, die Facharbeiter sind die Consumer-Threads, die Anzeigetafel das wxPython-Dashboard. In 13 Akten führt die Geschichte vom Fabrikstart (Initialisierung) über den Arbeitsalltag (Marker-Verarbeitung, Heartbeats, Crash-Recovery) bis zur Schließung (Graceful Shutdown). Codeausschnitte und Mermaid-Diagramme sind direkt eingebettet, sodass jede Analogie sofort auf die Implementierung zurückgeführt werden kann. Der Grund für dieses Format: Nebenläufige Architekturen mit fünf parallelen Pipelines, Thread-Überwachung und Browser-Automatisierung sind aus reiner API-Dokumentation schwer zu erschließen. Die Erzählform macht das *Warum* hinter den Entwurfsentscheidungen greifbar und gibt künftigen Entwicklern einen Einstieg, der über das *Was* der Referenzdokumentation hinausgeht.

7.2.2 EV-Testing Dashboards (Splunk & ADX)

Hintergrund Bei der Absicherung von Elektrofahrzeugen fallen große Mengen Telemetriedaten an. Dashboards machen diese zugänglich, indem sie Trends, Anomalien und Kennzahlen visualisieren Few, 2013. Das Projekt bestand aus zwei Teilen: der Überarbeitung bestehender Splunk-Dashboards und der Neuentwicklung eines ADX-Dashboards.

Aufgabenstellung

Teil 1: Splunk-Dashboard-Verbesserungen. Die bestehenden Artemis-Daten- und Connectivity-Dashboards in Splunk sollten überarbeitet und um neue Filtermöglichkeiten, Plattformunterstützung und verbesserte Benutzerführung erweitert werden. Diese Dashboards waren ein nützlicher Baustein für die EV-Dashboard-Sammlung des Betreuers, um die vorhandenen Splunk-Daten besser zu visualisieren.

Teil 2: EV-Testing Dashboard in Azure. Ein neues Dashboard in der Cloud-basierten Datenanalyseplattform ADX sollte von Grund auf entwickelt werden. Es basiert auf VVR-Daten und visualisiert Signale wie Ladezustand (SOC), Ladeverhalten, Geoposition und Backend-Kommunikation.

Ergebnisse Die Splunk-Dashboards erhielten Artemis-Plattform-Unterstützung, erweiterte Filter und verbesserte Connectivity-Darstellung (47 Konfigurationen). Eine Herausforderung war die

Token-Initialisierung: Dashboard Studio führt Abfragen sofort beim Laden aus, sodass fehlende Tokens zu *Waiting for Input* führen. Die Lösung – direkte Token-Zuweisung mit `defaultValue` statt Event-Handleern – wurde auf alle Dashboards angewendet und in der RDI-Schulung als Best Practice weitergegeben.

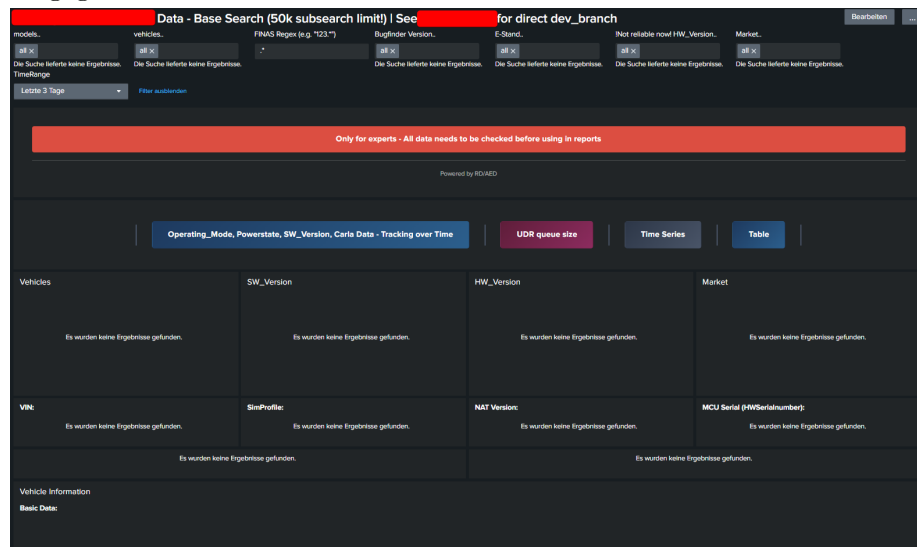


Abbildung 6: Splunk Artemis Data Dashboard mit Fahrzeug- und Softwarefiltern

Das EV-Testing Dashboard in ADX umfasst 35 Seiten mit 220 Visualisierungen und 145 Kusto-Abfragen. Es stellt historische Backend-Parameterdaten aus VVR dar: Kusto-Abfragen mappen Rohwerte auf lesbare Texte, visualisiert in farbcodierten Tabellen, Karten und Diagrammen. Die Bereiche: SOC-Monitoring, Ladeprozess-Analyse, Range Assistance (EVRA), Geolokation und Backend-Kommunikation. Integrierte Markdown-Dokumentation erklärt jede Seite.



Abbildung 7: EV-Testing Dashboard – Ladeprozess-Analyse mit farbcodierter Statustabelle und Geoposition

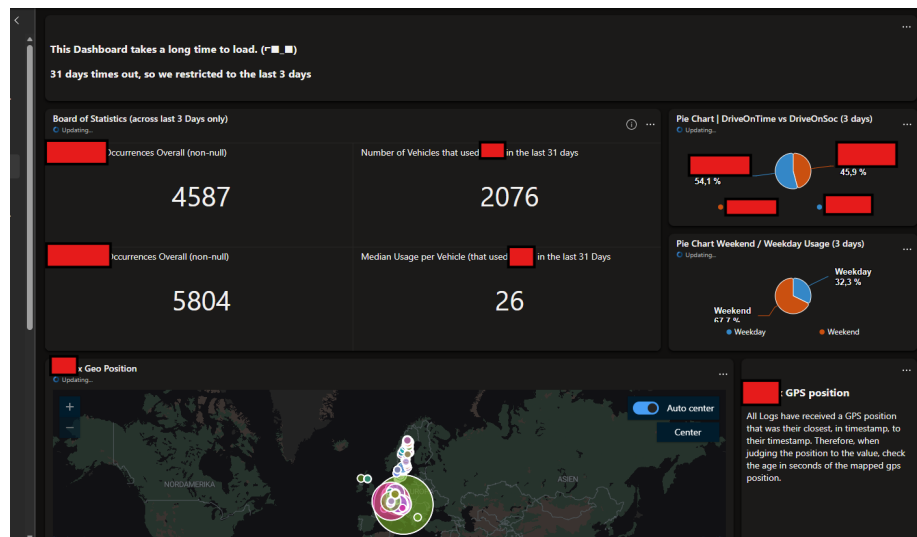


Abbildung 8: EV-Testing Dashboard – Flottenweite Feature-Statistiken mit geografischer Verteilung

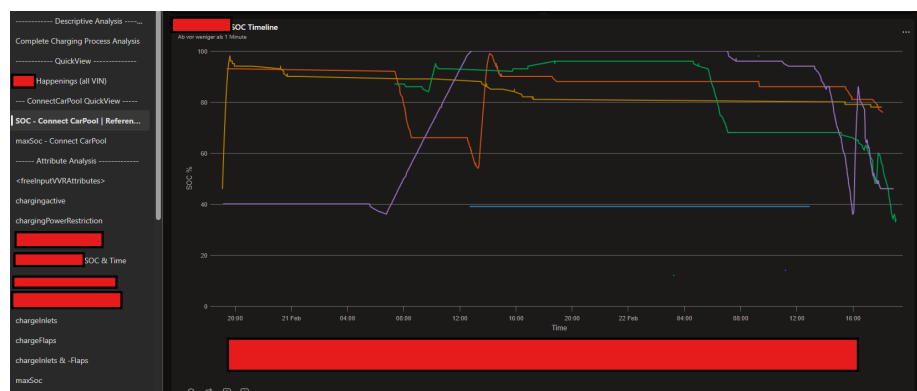


Abbildung 9: EV-Testing Dashboard – SOC-Timeline der Testfahrzeuge mit CarPool-Navigation und Attribut-Analyse

Nutzen für das Team Die Dashboards veränderten die tägliche Arbeitsweise: Fragen wie „Wie oft tritt ein Connectivity-Fehler seit dem letzten Softwarestand auf?“ erforderten zuvor manuelles Durchsuchen von Logdateien (Minuten bis Stunden). Mit den Dashboards: wenige Klicks, Echtzeit-Ergebnisse. Einsatzgebiete: Sprint-Reviews, Regressions-Identifikation, Defekt-Priorisierung und Entscheidungsvorlagen für Management. Das EV-Testing Dashboard wurde vom Betreuer als Vorlage für die abteilungsweite EV-Dashboard-Sammlung übernommen.

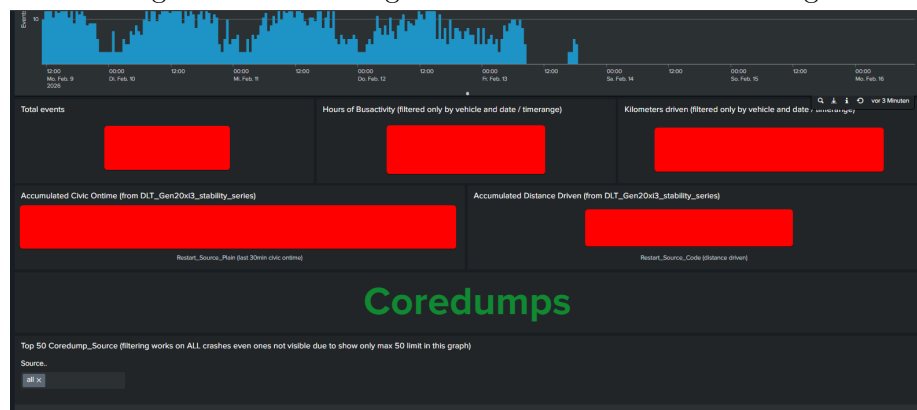


Abbildung 10: DLT Stability Dashboard mit feingranularer Auswertung

7.2.3 DLT-Log-Analyse-Pipeline (CarlaDLTAnalysis)

Hintergrund Zur Bewertung der Softwarequalität müssen große Mengen Diagnosedaten systematisch aufbereitet werden. Steuergeräte erzeugen kontinuierlich Diagnostic Log and Trace-Logs (DLT) AUTOSAR, 2017; COVESA, 2025 mit Systemereignissen, Fehlermeldungen und KPI-Markern. Diese Rohdaten müssen gefiltert und strukturiert werden, bevor sie auswertbar sind.

Aufgabenstellung Die Pipeline (Abbildung 11) war bereits im Einsatz. Meine Aufgabe: relevante DLT-Logzeilen recherchieren, daraus Java-Filter mit Regex-Extraktion entwickeln und sicherstellen, dass die Ausgabe ins MariaDB-Schema passt. Auf der Splunk-Seite erstellte ich Analyselogik und Dashboards.



Abbildung 11: Datenfluss der bestehenden DLT-Analyse-Pipeline. Eigener Beitrag: Java-Filter (Mitte) und Splunk-Dashboards (rechts)

Ergebnisse Mein Beitrag auf der Filterseite umfasste zwei zentrale Filter sowie kleinere Anpassungen und Fehlerbehebungen an bestehenden Filtern:

- **Stability-Filter** (DLT_Gen20xi3_Stability): Segmentiert DLT-Logs in 30-Minuten-Zeitfenster, abbildbar auf Reservierungen im Fahrzeugbuchungssystem, um gezielt Logdaten aus tatsächlichen Testzeiten zu filtern. Pro Fenster werden Head-Unit-On-time und Kilometerstand extrahiert – Normalisierungsbasis für Coredumps pro Stunde bzw. Kilometer (siehe Normalized Software Evaluation).
- **RMUI-Filter** (DLT_Gen20xi2_RMUIV1, 804 Zeilen): Extraktion von Remote-UI-Kennzahlen (CarPlay, Android Auto) aus DLT-Protokollen. Der Filter definiert 18 spezialisierte `DLTFilter`-Instanzen, konfiguriert über ECU-ID, App-ID, Context-ID und Payload-Muster. Für **CarPlay** werden u. a. iAP2-Authentifizierungsfehler, Session-Timeouts, Siri-Verbindungsprobleme und MFi-Initialisierungsfehler erfasst; für **Android Auto** Zertifikats-Validierungsfehler und Kommunikationsabbrüche. Pro Event wird eine 18-spaltige CSV-Zeile mit Fahrzeug-ID, Softwarestand, Zeitstempel und Payload erzeugt. Der Filter entstand in Zusammenarbeit mit einem RDI-Entwickler, der die relevanten Logzeilen definierte, während ich Filterimplementierung und Splunk-Integration übernahm.

Der Prozess war bei beiden Filtern gleich: DLT-Logzeilen recherchieren (ECU-ID, App-ID, Context-ID, Payload), als Java-Klasse nach `DLT_AnalysisBase` implementieren und CSV-Kompatibilität mit dem MariaDB-Schema sicherstellen. Beide Filter laufen in der täglichen CI/CD-Pipeline.

Auf der Analyseseite erstellte ich Splunk-Dashboards, die indexierte Daten in Visualisierungen überführen – Softwarestabilität und Connectivity über Sprints und Generationen hinweg.

7.2.4 Normalized Software Evaluation

Hintergrund Bei der Bewertung der Softwarequalität über verschiedene Fahrzeuggenerationen und Softwarestände hinweg stellt sich die Frage der Vergleichbarkeit: Ein Softwarestand, der auf 50 Fahrzeugen über 1.000 Stunden getestet wurde, produziert naturgemäß mehr Fehler als einer mit 10 Fahrzeugen und 100 Stunden. Absolute Fehlerzahlen sind daher wenig aussagekräftig.

Aufgabenstellung Ziel war die Entwicklung eines Bewertungsansatzes, der Softwarestabilität (Coredumps, Reboots) normalisiert nach Betriebszeit und gefahrenen Kilometern darstellt. Dies erforderte sowohl neue Filter in der internen Logdaten-Filter-Engine zur Extraktion der relevanten Kennzahlen aus DLT-Logs als auch ein Splunk-Dashboard zur Visualisierung.

Datenfluss Zwei Datenpfade laufen in MariaDB zusammen (Abbildung 12): Die CI/CD-Pipeline fragt über die API des Fahrzeugbuchungssystems 30-Minuten-Buchungsslots ab (zuvor

manuell per JSON-Request angelegt) und schreibt sie in MariaDB. Parallel extrahiert der Stability-Filter der Logdaten-Filter-Engine Stabilitätskennzahlen (Coredumps, Overtime, Kilometerstand) aus DLT-Logs. Eine Splunk-Abfrage führt beides zusammen und normalisiert die Crash-Häufigkeit nach Overtime und gefahrener Strecke.

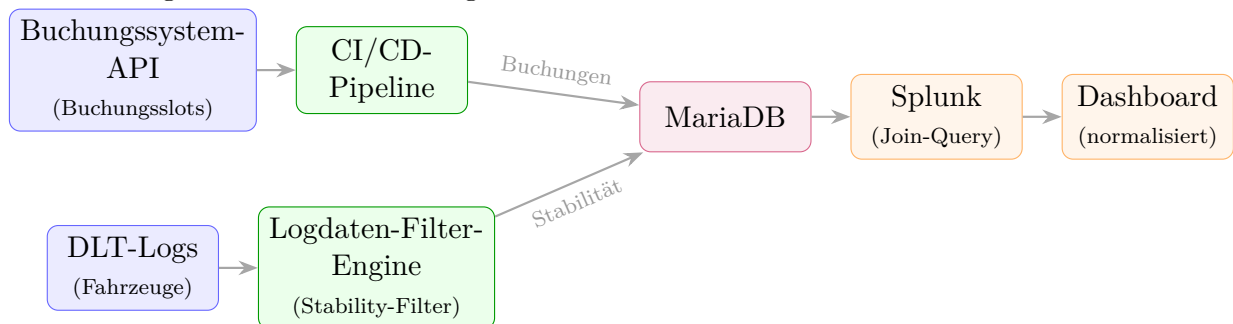


Abbildung 12: Datenfluss der Normalized Software Evaluation: Zwei Datenpfade konvergieren in MariaDB

Ergebnisse Auf der Filterseite wurde der bestehende Stability-Filter (DLT_Gen20xi3_Stability.java) um die Erfassung der CIVIC-Overtime (Betriebszeit in Sekunden) und des Kilometerzählers erweitert. Diese Werte werden neben den Coredump-Zählern als CSV-Spalten exportiert.

Das Splunk-Studio-Dashboard zeigt pro Sprint *CoredumpsPerHour* und *CoredumpsPerKm*, aufgeschlüsselt nach MB.OS-Version und Softwarezweig. Datenquelle ist die CI/CD-befüllte CSV-Lookup-Tabelle *NormalizedSwEvaluation.csv*. Ein Sortierschlüssel JJ.KW (z. B. 25.39 → 2539) sichert die chronologische Darstellung über Jahresgrenzen; Balkendiagramm und Drill-Down-Tabelle ermöglichen Sprint-Vergleiche.

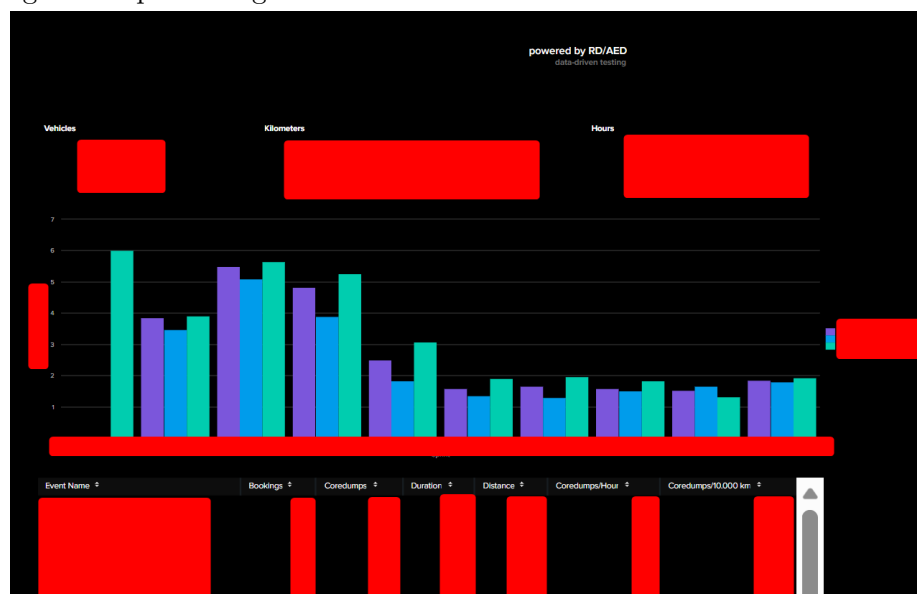


Abbildung 13: Normalized Software Evaluation – Splunk Studio Dashboard

Dieses Projekt bildete die Grundlage für die Schulung des RDI-Teams (siehe Abschnitt 7.2.5).

7.2.5 Splunk-Schulung RDI CarPlay-Team

Hintergrund Das RDI-Entwicklerteam in Bangalore war für die CarPlay-Qualitätssicherung zuständig, hatte jedoch keinen Zugang zur DLT-Analyse-Infrastruktur (Splunk, DLT-Filter, Dashboards).

Aufgabenstellung Ziel: Das RDI-Team befähigen, eigenständig DLT-Daten zu filtern und Splunk-Dashboards für CarPlay-Analysen zu nutzen, einschließlich neuer Filter und Dokumentation.

Ergebnisse Über mehrere Wochen fanden regelmäßige Schulungsmeetings per Videokonferenz statt (Zeitverschiebung Indien: +4:30h). Die Inhalte umfassten:

- Einführung in Splunk: Suche, Dashboards, Lookups und die Normalized Software Evaluation als Anwendungsbeispiel
- Aufbau und Funktionsweise der internen Logdaten-Filter-Pipeline
- Erstellung von CarPlay-spezifischen DLT-Filtern: Die Schulung basierte auf dem gemeinsam entwickelten RMUI-Filter (vgl. Abschnitt CarlaDLTAnalysis), der CarPlay-Events wie `mobileDeviceStatusChanged` mit den Zuständen `RUI_ACTIVATION` und `RUI_DEACTIVATION` aus den DLT-Logs extrahiert
- Anpassung des Normalized-Software-Evaluation-Dashboards für CarPlay-Metriken, sodass CarPlay-spezifische Coredumps und Session-Abbrüche analog zur Gesamtsoftware normalisiert dargestellt werden
- Troubleshooting bei lokaler Einrichtung der Filter-Engine (Multi-Baureihen-Konfiguration)

Nach der Schulung erstellte der Entwickler eigenständig ein Splunk-Dashboard für die Remote-UI-Analyse (CarPlay/Android Auto), das auf den von mir entwickelten Filtern basiert. Abbildung 14 zeigt die Rückmeldung des Hauptansprechpartners aus dem RD-Indien-Team zum Abschluss meines Praktikums.

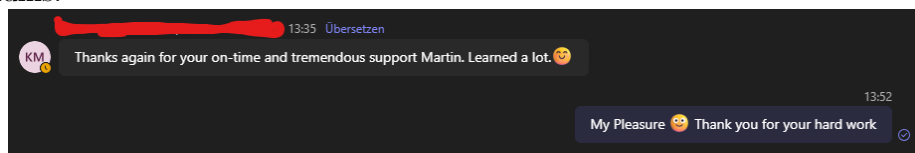


Abbildung 14: Danksagung des Hauptansprechpartners aus dem RD-Indien-Team zum Abschluss meines Praktikums

7.2.6 Logger-Adapterkabel

Hintergrund Für die Diagnose und das Logging bestimmter Fahrzeugsignale (Inlet-CAN und Energy-CAN) in Telematik-Fahrzeugen werden spezielle Adapterkabel benötigt, die den Datenlogger mit den entsprechenden CAN-Bussen verbinden. Dedizierte Adapterkabel waren im Team bislang nicht vorhanden; stattdessen wurden vereinzelt alte Kabel aus früheren Projekten wiederverwendet. Die Alternative – ein zweiter Logger für ca. 16.000 € – war unwirtschaftlich, wenn das Problem durch selbstgebaute Adapterkabel für ca. 40 € Materialkosten lösbar war.

Aufgabenstellung Es sollten neue Logger-Adapterkabel hergestellt werden, die den Datenlogger mit den Inlet- und Energy-CAN-Bussen in verschiedenen Versuchsfahrzeugen verbinden. Die Herstellung erforderte Mikrolöten und Crimpen von Steckverbindern nach den Spezifikationen der X2E-Logger. Insgesamt waren 40 Einzeladapter (jeweils 20 empfangende und 20 sendende Stecker) zu fertigen, die zu 10 vollständigen Kabelsätzen für die X2E-Logger zusammengesetzt wurden. Als Steckverbinder kamen unter anderem 20-polige SCSI-Buchsen am HQ-Großstecker (CAN A, Pins 21–30) sowie Flachstecker für die MESS1/MESS2-CAN-Kanäle zum Einsatz.

Ergebnisse Im Labor wurden über mehrere Wochen hinweg zwei Generationen von Logger-Adapterkabel-Sätzen hergestellt:

- **Generation 1:** Grundlegende Inlet- und Energy-CAN-Kabel für die Standardkonfiguration
- **Generation 2 (EIA V2):** Verbesserte Version, bei der die Kabelführung so umgestaltet wurde, dass der Logger-Einbau im Kofferraum ohne Demontage der Verkleidung möglich ist, und eine zusätzliche X2E-Kanalerweiterung das gleichzeitige Logging von Inlet- und Energy-CAN auf separaten Kanälen ermöglicht



Abbildung 15: Arbeitsplatz für den Kabelbau mit Löt- und Crimpwerkzeug

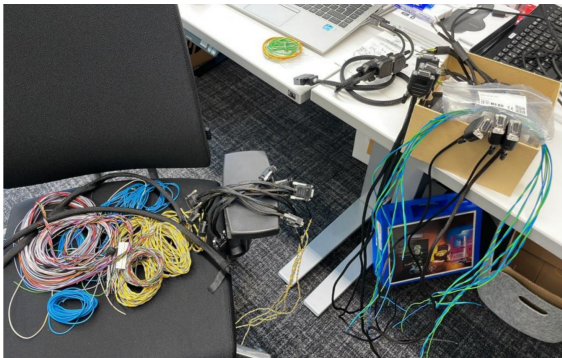


Abbildung 16: Rohkabelsätze vor dem Löt- und Crimpen



Abbildung 17: Halbfertige Kabelsätze während der Produktion

Die fertigen Kabelsätze wurden in den Versuchsfahrzeugen der Flotte installiert und getestet. Begleitend entstand eine Dokumentation (*HowToEnableInletAndEnergyCanInTelematicCars.pdf*), die den Einbau und die Konfiguration der Logger-Firmware für nachfolgende Praktikanten beschreibt.

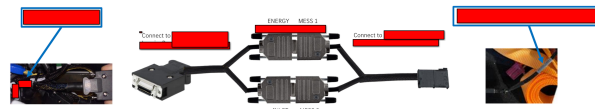


Abbildung 18: Fertiger Kabelsatz mit zugehörigem Fahrzeug-Anschlusschema

Diese Hardwarearbeit war für mein Gesamtverständnis der Datenerfassungskette essenziell: Die Qualität der Dashboards und DLT-Analysen, die ich in den anderen Projekten erstellte, hängt direkt davon ab, dass die physische Verbindung zwischen Fahrzeug-CAN-Bus und Datenlogger zuverlässig funktioniert. Das Verständnis dieser Hardware-Ebene hat meine Fähigkeit zur Fehlerdiagnose in der gesamten Pipeline – vom Kabel über den Logger bis zum Splunk-Dashboard – deutlich verbessert.

7.2.7 Nebenbei Gelerntes

Neben den eigentlichen Projekten wurden im Verlauf des Praktikums zahlreiche Fähigkeiten und Kenntnisse erworben, die nicht einem einzelnen Projekt zuzuordnen sind, aber in ihrer Gesamtheit das Verständnis für die Systemlandschaft des Connected Car entscheidend vertieft haben:

- **Fahrzeugdiagnose:** Flashen von Steuergeräten in regelmäßigen Flash-Hell-Sessions (Pfadstruktur mit bis zu 9 Containern und Variantenauswahl), Variationcoding (FiNAS → Codierungsfile → Flash → Systemaktivierung → Kurztest), ZenZefi Database Reset, SecOC/Tickrate-Inbetriebnahme (mit und ohne VSM, ECU-Zeitsynchronisation) und VeDoc-Datenkartenverwaltung (SA-Codes, Steuergeräte-Übersicht). Nutzung von Monaco für Diagnosekommandos an Steuergeräte.
- **VVR/Fleet Builder Recherche:** Empirisches Reverse Engineering der Refresh-Zeiten zwischen VVR, Fleet Builder und dem Fahrzeugbuchungssystem. Durch systematische Messungen ermittelte ich die Verzögerungskette nach einem Flash: ca. 5 Min bis VVR, ca. 30 Min bis zur korrekten Anzeige in allen Systemen. Diese Erkenntnisse flossen in die automatische Softwarestand-Erkennung des Test Session Tools ein.

- **Fahrzeuginformationssystem-API:** Nutzung der API des Fahrzeuginformationssystems zur programmatischen Erstellung von Sprint-Events mit Custom-Filtern – ein Seitenprojekt im Rahmen der Normalized Software Evaluation. Dabei vertiefte ich den Umgang mit Swagger-API-Dokumentation, Application-JSON-Bodies und dokumentenorientierten Datenbanken.
- **Hochvolt-Sicherheit:** Absolvierung der HV-Sensibilisierungsschulung (HV2) für den sicheren Umgang mit Hochvolt-Antriebssystemen in Elektrofahrzeugen.
- **Netzwerk und Connectivity:** Verständnis der Fahrzeugvernetzung über TCU, HU und CAN-Busse. Analyse von Backend-Log-Zeiten, OTAR-Logs und Service-Aktivierung.

In Kombination haben diese Kenntnisse meine Fähigkeit zur Fehleranalyse über die gesamte Kette – von der physischen Hardware über die Steuergerätesoftware bis zum Backend – aufgebaut, die in den Hauptprojekten (Dashboards, DLT-Filter, Test Session Tool) unmittelbar zum Tragen kam.

8 Selbstreflexion & Lernzielanalyse

8.1 Lessons Learned (Soft Skills)

8.1.1 Zwei Umgebungen, eine Person

Im Praktikum habe ich eine klare Erkenntnis gewonnen: Ich arbeite am produktivsten, wenn Kommunikation explizit und strukturiert erfolgt – Aufgaben mit direkter Zuweisung, Deadline und Priorität. Implizite Erwartungen oder indirekte Hinweise in Gruppenchats funktionieren für mich schlechter – nicht aus mangelndem Willen, sondern weil ich Kontextinformationen über explizite Kanäle aufnehmen statt aus informellen Signalen.

Das Praktikum begann mit 50% CarPool-Management und 50% Toolentwicklung unter meinem Betreuer Martin Russold. Zwei Wochen später übernahm ich zusätzlich die *Normalized Software Evaluation* – die Aufteilung wurde 40/30/30.

Damit arbeitete ich in drei Umgebungen mit unterschiedlichen Kommunikationskulturen:

Umgebung 1: Toolentwicklung (Martin Russold). Klare Aufgabenstellung, direktes Feedback, regelmäßiger Austausch. Ich wusste, was erwartet wurde, und konnte liefern. In diesem Bereich entstand das Test Session Tool mit 174 Quelldateien, das EV-Dashboard, die Splunk-Dashboards und die DLT-Filterentwicklung. Mein Betreuer bewertete Leistung und Engagement durchgehend positiv.

Umgebung 2: Normalized Software Evaluation. Nach der initialen Aufgabenstellung folgten rund vier Monate ohne Zwischenfeedback des Projektverantwortlichen. Ich baute die Dashboards eigenständig auf – Stabilitätsdaten normalisiert nach Zeit und Kilometern als DLT-Filter und Splunk-Studio-Dashboards. Als kurzfristig Ergebnisdruck entstand, fehlte auf beiden Seiten ein Bezugsrahmen zum Arbeitsstand – vermeidbar bei regelmäßigem Austausch. Das eigenständige Arbeiten war wertvoll für die Selbstorganisation, zeigte mir aber, dass proaktives Einfordern von Zwischenfeedback eine Verantwortung ist, die ich künftig bewusster wahrnehmen werde.

Umgebung 3: CarPool-Management. Aufgaben wurden über ein gemeinsames E-Mail-Postfach verteilt, koordiniert durch eine leitende Praktikantin. In sechs Monaten kam es zu keinem direkten bilateralen Austausch. Kommunikation erfolgte, wenn überhaupt, indirekt über einen Gruppenchat – der de facto kaum genutzt wurde. Es gab keine direkte Adressierung, keine Deadlines, keine Prioritäten. De facto fand keine Kommunikation statt.

Ich kommunizierte frühzeitig, was ich benötigte: direkte Ansprache, klare Zuweisung, idealerweise Kalendereinträge. Mein Lösungsvorschlag wurde erst Wochen später indirekt abgelehnt – selbst dies ein Beispiel der indirekten Kommunikationsstruktur. Nach sechs Wochen erhielt ich die Rückmeldung, proaktiver zu sein. Ich setzte dies um: räumliche Nähe, aktive Gesprächsteilnahme, Hilfsangebote. Die Wirkung blieb aus. Nach drei Monaten wurde die Situation ausschließlich auf mein Verhalten zurückgeführt, ohne umsetzbare Vorschläge – obwohl ich darum bat.

Rückblickend spielte auch das Fehlen formaler Feedbackprozesse eine Rolle: Ohne strukturierte Zwischengespräche oder Anlaufstellen wurde die Situation erst spät adressiert.

Auf meiner Seite erkenne ich Versäumnisse: Ich hätte *früher eskalieren* sollen – bereits in der ersten Woche, in der die vereinbarte Kommunikationsform nicht funktionierte. Ich hätte wiederholter vermitteln müssen, *warum* direkte Zuweisung Voraussetzung ist, nicht Präferenz – eine einmalige Erklärung reicht nicht. Zudem nahm ich informelle Teamevents selten wahr, weil Einladungen über Mailinglisten für mich nicht als persönliche Ansprache erkennbar waren. Diese hätten die fehlenden informellen Beziehungen aufbauen können – dasselbe Muster auf allen Ebenen: Ohne explizite Kommunikation erreichten mich auch gut gemeinte Angebote nicht.

Der Kontrast weist über meine Situation hinaus: Dieselbe Person, die eigenständig ein Tool mit über hundert Quelldateien entwickelte, eine Schulung initiierte und Dashboards für die Abteilung aufbaute, wurde in einem anderen Bereich als unzureichend proaktiv eingestuft. Dieser Widerspruch deutet auf die entscheidende Rolle der Passung zwischen Arbeitsumgebung und Kommunikationsbedürfnissen hin – ein Faktor, der für jede Teamzusammensetzung relevant ist.

8.1.2 Was funktioniert hat

Aus den funktionierenden Bereichen nehme ich konkrete Lektionen mit.

Fehlerkultur und die Preflight Checklist. In der Luftfahrt hat das Konzept des *Crew Resource Management* (CRM) die Unfallraten drastisch gesenkt Helmreich und Merritt, 2000. Der Kern: Hierarchie darf nie dazu führen, dass ein Co-Pilot einen Fehler des Kapitäns nicht anspricht. Im Fahrzeugtest habe ich gelernt, Defekte sachlich und reproduzierbar im zentralen Defect-Management-System zu dokumentieren – ohne Schuldzuweisung, aber ohne Weglassen.

Ein zweites Luftfahrtkonzept übertrug ich direkt in meine Software: die *Preflight Checklist*. Das Test Session Tool zeigt vor Sessionstart alle Parameter an – Fahrzeug-ID, Softwarestand, Testprofile, Verbindungsstatus. Der Tester bestätigt explizit, bevor die Session startet. Da eine Session mehrere Stunden dauert, sparen die wenigen Sekunden Preflight Check im Fehlerfall Stunden.

Zielgruppen überzeugen. Bei Präsentationen merkte ich schnell, dass technische Tiefe nicht immer überzeugt. Das DISC-Verhaltensmodell Marston, 1928 half als Orientierung: Der *rote* (dominante) Typ will Ergebnisse, der *gelbe* (initiative) begeistert sich für Möglichkeiten, der *grüne* (stetige) braucht Sicherheit, der *blaue* (gewissenhafte) verlangt Daten. Martin Russold konnte ich in Kusto-Queries einsteigen; einem Teamleiter musste ich mit dem Geschäftswert anfangen – *wie viele Stunden spart das pro Woche?* Diese empfängergerechte Kommunikation war eine der wichtigsten Lektionen.

Sprint vs Marathon. Anfangs wollte ich möglichst viel in kurzer Zeit liefern. Kurzfristig funktionierte das, führte aber zu schwankender Qualität: Bei paralleler Arbeit an EV-Dashboard, DLT-Filter und Splunk-Schulung fehlte Zeit für Tests – ein Dashboard-Fehler fiel erst im Review auf. Ein Praktikum über sechs Monate ist kein Sprint, sondern ein Marathon. Ich lernte, realistisch zu priorisieren und auch *Nein* zu sagen. Das Ergebnis: nicht weniger, sondern konsistenterer Output.

Informationen beschaffen und am Ball bleiben. Im Connected-Car-Bereich ändern sich Technologien schnell. Drei Strategien: Erstens, Confluence und Wikis systematisch lesen. Zweitens, Kollegen direkt fragen – eine Teams-Nachricht ist oft effektiver als eine Stunde Recherche. Drittens, eigene Dokumentation pflegen – Verstandenes in Confluence festhalten.

8.1.3 Eigeninitiative: Von der Normalisierung zur RDI-Schulung

Ein Beispiel für Eigeninitiative entstand aus der Normalized Software Evaluation – jenem Projekt, in dem ich vier Monate ohne Anleitung arbeitete.

Nach Fertigstellung der Normalisierungs-Dashboards erhielt ich die Aufgabe, ähnliche Analysen für CarPlay-Logs zu erstellen. Dabei wurde klar: Ein einzelnes Dashboard ist eine Einmallösung – wenn ich gehe, veraltet es. Nachhaltiger war es, das Entwicklerteam zur eigenständigen Analyse zu befähigen.

Ich kontaktierte den RemoteUI-Funktionsverantwortlichen und schlug einen Tausch vor: Schulung in Datenanalyse und Splunk gegen Zugang zur CarPlay-Log-Dokumentation. Beide Seiten profitierten: Das RDI-Team wurde zur eigenständigen DLT-Analyse befähigt, und ich erhielt die fachliche Grundlage für meine Arbeit – eine Zusammenarbeit, die über mein Praktikum hinaus wirkt.

Niemand hatte mir das aufgetragen. Ich identifizierte ein systemisches Problem – personengebundenes Wissen ist fragil – und wählte die nachhaltige Lösung. Diese Initiative entstand nicht trotz fehlender Anleitung, sondern weil ich gelernt hatte, ohne sie zu arbeiten.

8.1.4 Feedbackkultur

Im Praktikum erlebte ich unwirksames Feedback – und lernte von meinem Betreuer, wie wirksames funktioniert.

Unwirksame Feedbackmuster. In einigen Bereichen folgte Feedback einem Muster: ohne vorherige Abstimmung, als Tatsache statt als Wahrnehmung formuliert, ohne umsetzbare Handlungsoptionen. Einmal bestand die Rückmeldung aus einer Buchempfehlung ohne Bezug zu konkretem Verhalten; ein anderes Mal sollte ich *proaktiver* sein, ohne Definition von Proaktivität in diesem Kontext. Rückmeldungen kamen punktuell nach langen Pausen, sodass der Bezugsrahmen fehlte. Ich hatte mehrfach Zwischenfeedback angefordert – die Antwort war jedes Mal sinngemäß „Danke, schaue ich mir an“, ohne dass eine Rückmeldung folgte. Die Lektion: Wenn trotz aktiver Nachfrage kein Feedback und kein Fortschrittsreview erfolgt, kann die eigene Seite wenig ausrichten. Ausbleibende Rückmeldung ist keine Zustimmung und kein Desinteresse – aber ohne Feedback fehlt jede Steuerungsmöglichkeit.

Wirksame Feedbackprinzipien. Martin Russold vermittelte mir durch Vorbild und Gespräch drei Prinzipien:

1. **Konsens:** Vor Feedback fragen, ob der Empfänger offen dafür ist. Ungefragtes Feedback wird leicht als Bewertung statt Unterstützung wahrgenommen.
2. **Subjektivität:** Feedback beschreibt die eigene Wahrnehmung, nicht die Realität. „Ich habe den Eindruck, dass...“ statt „Du bist...“ – ein Angebot zur Reflexion, kein Urteil.
3. **Umsetzbarkeit:** Jede Rückmeldung enthält eine konkrete Handlungsoption. Ohne sie bleibt Feedback ein Vorwurf.

Diese Prinzipien wende ich seitdem selbst an. Ich lernte sie nicht aus einem Lehrbuch, sondern von meinem Betreuer – und indem ich zuerst erlebte, wie es sich anfühlt, wenn sie fehlen.

8.1.5 Was ich mitnehme

Simon Sinek beschreibt in *The Infinite Game* ein Modell der US Navy SEALs Sinek, 2019: Bei der Teamauswahl zeichneten sie ein Koordinatensystem mit *Performance* und *Trust* als Achsen. Die entscheidende Erkenntnis: Die SEALs bevorzugen konsequent mittlere Kompetenz bei hohem Vertrauen gegenüber hoher Kompetenz bei niedrigem Vertrauen. Fachliches lässt sich trainieren – Vertrauen kaum.

Darin erkannte ich meine Erfahrung: *Wie* jemand arbeitet ist mindestens so wichtig wie *was* geliefert wird. Wo Vertrauen und offene Kommunikation vorhanden waren, leistete ich meine beste Arbeit. Wo beides fehlte, konnte auch Eigeninitiative die Lücke nicht schließen.

Drei Erkenntnisse nehme ich mit:

- **Selbstvertretung erfordert Ausdauer und Akzeptanz.** Bedürfnisse klar formulieren und Lösungen vorschlagen ist notwendig, aber nicht immer hinreichend. Nicht jedes Umfeld ist

anpassbar – wo die Passung trotz Bemühungen fehlt, ist das ein strukturelles Signal. Künftig werde ich bereits im Onboarding Kommunikationsstrukturen klären und Bedürfnisse schriftlich dokumentieren.

- **Umgebung und Leistung sind untrennbar.** Dieselbe Person kann in einem strukturierten Umfeld hochproduktiv und in einem impliziten deutlich weniger wirksam sein. Passung ist keine Sonderbehandlung, sondern Produktivitätsvoraussetzung. Künftig werde ich bei der Arbeitsplatzwahl bewusst auf Kommunikationsstrukturen achten.
- **Energie gezielt einsetzen.** Wenn Zusammenarbeit trotz Lösungsversuchen und Eskalation nicht funktioniert, ist es sinnvoller, die Energie auf funktionierende Beziehungen zu konzentrieren. Konkret: nach zwei ergebnislosen Anläufen frühzeitig eine dritte Person als Vermittler einbeziehen.

8.2 Lessons Learned (Technical Skills)

8.2.1 Requirements Engineering und UX-Design gehen Hand-in-Hand

Am Test Session Tool (174 Python-Quelldateien, über 1000 Tests) lernte ich, dass technische Anforderungen und Benutzererfahrung parallel entstehen müssen. Der erste GUI-Entwurf war funktional korrekt, aber schwer bedienbar: sechs separate Konfigurationsdialoge, Verarbeitungsstatus nur in der Konsole. Durch Nutzerfeedback entstand ein Redesign: ein einziger Preflight Check-Dialog und eine Statusleiste mit Fortschrittsanzeige im Hauptfenster. Die Producer-Consumer-Pipeline mit fünf unabhängigen Queues und automatischem Thread-Recovery entstand aus der Anforderung, dass langsame Trace-Extraktion (bis 5 Min) die Videoerfassung (3 s) nicht blockieren darf – eine Architekturentscheidung, die direkt aus dem Nutzungskontext folgte. Die Tester nutzten das Tool anschließend eigenständig. Lektion: Ein Feature, das niemand benutzt, ist wertlos – ob es benutzt wird, entscheidet sich an der Schnittstelle zum Nutzer.

8.2.2 Netzwerktopologie

CAN, DLT, TCU und HU waren anfangs abstrakte Abkürzungen. Durch tägliche Arbeit mit Logdaten wurden sie konkret und veränderten meine Arbeitsweise. Vorher betrachtete ich Defekte isoliert: *Die App stürzt ab*. Mit Verständnis der Fahrzeugnetzwerk-Topologie konnte ich denselben Defekt im Systemkontext analysieren: *Die App stürzt ab, weil die TCU nach einem OTA-Update eine veränderte Nachrichtenfrequenz auf dem CAN-Bus sendet, die der HU-Prozess nicht erwartet*. Die Fähigkeit, in DLT-Traces die Kommunikation schichtübergreifend nachzuvollziehen, war entscheidend für meine Filter und Dashboards. Die Lektion reicht über das Automobil hinaus: Wer nur die Softwareschicht versteht, debuggt Symptome; wer auch die darunterliegende Infrastruktur kennt, findet Ursachen.

8.2.3 Fehlerbereinigung und Defekt-Reproducing

Die systematische Reproduktion von Softwaredefekten war eine der anspruchsvollsten Fähigkeiten. Ein Defekt, der nicht reproduzierbar ist, kann nicht gefixt werden. Methodisches Vorgehen: Umgebungsbedingungen dokumentieren, Schritte minimieren, Variablen isolieren. In der DLT-Analyse hieß das: tausende Logzeilen auf den relevanten Zeitstempel eingrenzen und die Kausalkette rückwärts verfolgen. Diese Fähigkeit – von Symptom zu Ursache – ist universell anwendbar und war eine der wertvollsten technischen Lektionen.

8.2.4 Sicherheitsbewusstsein in vernetzten Systemen

Softwarequalität in vernetzten Fahrzeugen hat eine Sicherheitsdimension über klassische Fehler hinaus. Im Verlauf des Praktikums erlebte ich, wie OTA-Updates die gesamte Flotte betreffen – ein fehlerhaftes Update kann Funktionsausfälle bis hin zu sicherheitsrelevanten Situationen verursachen. Konkret: Meine Dashboards dienen als Entscheidungsgrundlage für Softwarefreigaben – eine falsche Darstellung könnte zur Freigabe instabiler Software führen. Im Test Session Tool

war die korrekte Zuordnung von Traces, Videos und Defektbeschreibungen essenziell, da Fehlzuordnungen die Analyse in die falsche Richtung lenken. Diese Erfahrungen vermittelten mir, dass Softwareentwicklung in sicherheitsrelevanten Domänen besondere Sorgfalt erfordert.

9 Liste der eingesetzten Software-Werkzeuge

Die folgende Tabelle listet alle wesentlichen Software-Werkzeuge, die im Verlauf des Praktikums eingesetzt wurden – von Entwicklungsumgebungen über Analyse- und Visualisierungstools bis hin zu fahrzeugspezifischen Diagnosewerkzeugen.

Name des Werkzeuges	Zweck und Verwendung
VS Code	Hauptentwicklungsumgebung (Python, LaTeX, allgemein)
IntelliJ IDEA	Java-Entwicklung (DLT-Filter der Logdaten-Filter-Engine)
PyCharm	Python-Entwicklung und Debugging
Python 3.11 / 3.12	Test Session Tool, Automatisierungsskripte
Java	DLT-Filterentwicklung (interne Logdaten-Filter-Engine)
Splunk	Big Data Visualisierung, Dashboard-Entwicklung
Azure Data Explorer (Kusto)	EV-Testing Big Data Analytics
wxPython	GUI-Framework (Test Session Tool)
OpenCV	Videoaufnahme und -verarbeitung
Playwright	Browser-Automatisierung (VAM, ADX, API)
Piper TTS	Offline-Sprachbenachrichtigungen
Pandas / NumPy	Datenverarbeitung und -analyse
MariaDB	Datenspeicherung (DLT-Pipeline)
Pytest	Testautomatisierung
Git / Azure DevOps	Versionskontrolle
Confluence / Jira	Dokumentation und Aufgabenverwaltung
Interne Logdaten-Filter-Engine	DLT-Log-Filterung und -Analyse
Monaco	Fahrzeugdiagnose
Internes Fahrzeugbuchungssystem	Fahrzeugbuchung und Fleet Management
Gemini API	KI-gestützte Defektbeschreibung (DefectAI-Modul)
PyInstaller	Standalone-Build des Test Session Tools als <code>.exe</code>
VeDoc	Fahrzeugdatenkarten- und Konfigurationsverwaltung
ZenZefi	Steuergeräte-Datenbank-Reset bei Konfigurationsproblemen
Fleet Builder / VVR (Virtual Vehicle Representation)	Fahrzeugflotten-Übersicht und Softwarestand-Tracking
X2E Logger	Hardware-Datenlogger für CAN-Bus-Aufzeichnung
L ^A T _E X (KOMA-Script)	Erstellung dieses Praktikumsberichts

Tabelle 2: Eingesetzte Software-Werkzeuge

10 Themenvorschläge für Bachelorarbeiten

#	Themenbeschreibung
1.	Head-Unit-Videostream als Eingabe für KI-gestützte Defektanalyse Bei der Erstellung von Defekt-Tickets im Fahrzeugtest wird der Infotainment-Bildschirm derzeit über eine externe USB-Kamera aufgenommen. Eine qualitativ hochwertigere Alternative wäre die direkte Nutzung des Videosignals, das die Head Unit an die Fahrzeugdisplays sendet. Vorarbeiten hierzu existieren bereits, scheiterten jedoch an der Verschlüsselung des Signals, die für bestimmte Anwendungen (z. B. Streaming-Dienste, DRM-geschützte Inhalte) aktiv ist. Ziel der Arbeit wäre die Untersuchung, unter welchen Bedingungen der unverschlüsselte Videostream abgegriffen werden kann, und die Nutzung des Streams in zwei Anwendungsfällen: erstens als hochauflösende Videoanlage bei manuell erstellten Defekt-Tickets und zweitens als Eingabe für die KI-gestützte Defektbeschreibung im DefectAI-Modul des Test Session Tools, wobei zu evaluieren wäre, ob die höhere Bildqualität die automatisierte Beschreibung messbar verbessert.
2.	Pydantic-basierte Pipeline zur automatisierten Defekt-Zuordnung Aktuell werden im Test Session Tool erkannte Defekte manuell den zuständigen Fachbereichen zugeordnet. Diese Zuordnung erfordert Domänenwissen über die Organisationsstruktur und die jeweiligen Verantwortlichkeiten – Wissen, das sich häufig ändert und für neue Teammitglieder schwer zugänglich ist. Ziel der Arbeit wäre der Entwurf eines zusätzlichen Pipeline-Consumers im ResultConsumer, der auf Basis von Pydantic-Modellen die Metadaten aller zuweisbaren Defect-Management-Bereiche strukturiert erfasst und eingehende Defekte anhand ihrer Merkmale (betroffenes Steuergerät, Fehlerkategorie, Softwarestand) automatisiert auf den passenden Bereich abbildet. Durch die maschinenlesbare Pydantic-Schemadefinition ließe sich die Zuordnungslogik dynamisch an Organisationsänderungen anpassen, ohne den Quellcode zu modifizieren.
3.	Anomalie-Erkennung in DLT-Stabilitätsdaten mittels statistischer Methoden Die Normalized Software Evaluation liefert pro Sprint normalisierte Stabilitätskennzahlen (Coredumps pro Betriebsstunde, Coredumps pro 10.000 km). Derzeit werden Regressionen manuell durch visuellen Vergleich der Balkendiagramme erkannt – ein Prozess, der bei steigender Anzahl von Softwarezweigen, Fahrzeuggenerationen und Sprintkadenz nicht skaliert. Ziel der Arbeit wäre die Entwicklung eines automatisierten Anomalie-Erkennungssystems, das auf Basis historischer Stabilitätsdaten aus der MariaDB-Pipeline statistische Abweichungen identifiziert und proaktiv Warnungen generiert, wenn ein neuer Softwarestand signifikant von der Baseline abweicht. Die Arbeit würde die bestehende Splunk-Infrastruktur um einen Alert-Mechanismus erweitern und evaluieren, welche statistischen Verfahren (z. B. z-Score, IQR, CUSUM) für die spezifischen Verteilungseigenschaften von Crash-Daten am besten geeignet sind.

11 Fazit und Ausblick

Das Praxissemester bei Mercedes-Benz in der Abteilung Pre-Integration Telematik & Connected Services war intensiv und lehrreich. Die Kombination aus CarPool-Management und technischen Projekten bot ein vielseitiges Umfeld für fachliche und persönliche Weiterentwicklung.

Besonders wertvoll war die eigenverantwortliche Arbeit an Projekten mit direktem Mehrwert: Test Session Tool, EV-Dashboards und DLT-Pipeline-Erweiterungen bleiben über das Praktikum hinaus im Einsatz.

Die in Abschnitt 2 formulierten Lernziele konnten im Verlauf des Praktikums erreicht werden:

1. **Einblick in die Arbeitsweise eines großen Automobilherstellers:** Durch CarPool-Management und sprintbasierte Zusammenarbeit mit Teams in Sindelfingen und Bangalore habe ich Prozesse und Entscheidungsstrukturen eines Konzerns mit über 160.000 Mitarbeitenden kennengelernt.
2. **Praktische Softwareentwicklung im professionellen Umfeld:** Das Test Session Tool (174 Quelldateien, 1000+ Tests, Standalone-Builds) wurde eigenständig entwickelt, iterativ verbessert und im Testbetrieb eingesetzt – eigene Software im realen Kontext entwickelt, getestet und ausgeliefert.
3. **Umgang mit großen Datenmengen:** Die Arbeit mit Splunk und ADX (EV-Testing Dashboard mit 145 Kusto-Abfragen) vermittelte Erfahrung in Verarbeitung und Visualisierung großer Datenmengen.
4. **Fahrzeugtechnik und -diagnose:** Flashen, Logger-Kabelbau, HV-Schulung und tägliche Arbeit mit DLT-Traces und Monaco bauten ein Grundverständnis für Fahrzeugvernetzung und -diagnose auf.
5. **Zusammenarbeit im Team:** Von strukturierter Zusammenarbeit mit dem Betreuer über die eigeninitiierte RDI-Schulung bis zu den in Abschnitt 8 reflektierten Herausforderungen – gerade die Kontraste zeigten, wie stark Kommunikationsstrukturen die Produktivität beeinflussen.

Diese Erfahrungen bilden eine solide Basis für das weitere Studium und den Berufseinstieg.

Ausblick Die Werkzeuge und Dashboards werden weiterhin genutzt. Das Test Session Tool bietet Potenzial für KI-erweiterte Analyse und zusätzliche Datenquellen. Die Erfahrungen fließen in mögliche Bachelorarbeitsthemen ein (Abschnitt 10).

12 Danksagung

Dieses Praktikum wäre ohne die Unterstützung mehrerer Personen nicht in dieser Form möglich gewesen.

Mein besonderer Dank gilt meinem Betreuer Martin Russold, der mich während des gesamten Praktikums fachlich und persönlich begleitet hat. Er gab mir die Freiheit, das Test Session Tool und die Dashboards eigenverantwortlich zu gestalten, stand gleichzeitig für regelmäßiges Feedback bereit und hat mir durch sein eigenes Verhalten gezeigt, wie wirksame Kommunikation und Feedbackkultur im Arbeitsalltag aussehen.

Ebenso danke ich dem gesamten Team der Abteilung Pre-Integration Telematik & Connected Services für die kollegiale Zusammenarbeit und die Bereitschaft, Wissen und Erfahrungen zu teilen.

Der Hochschule Reutlingen, insbesondere der Fakultät Informatik, danke ich für die Organisation und Betreuung des Praxissemesters.

Literaturverzeichnis

- AUTOSAR. (2017). Specification of Diagnostic Log and Trace [AUTOSAR Classic Platform, Release 4.3]. Verfügbar 16. Februar 2026 unter https://www.autosar.org/fileadmin/standards/classic/4-3/AUTOSAR_SWS_DiagnosticLogAndTrace.pdf
- COVESA. (2025). *Diagnostic Log and Trace (DLT) Protocol*. Verfügbar 16. Februar 2026 unter <https://github.com/COVESA/dlt-daemon>
- Few, S. (2013). *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring* (2. Aufl.). Analytics Press.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software* [Insbesondere das Producer-Consumer-Pattern (Kap. 5), das die Architektur des Test Session Tools prägt]. Addison-Wesley.
- Helmreich, R. L., & Merritt, A. C. (2000). *Culture at Work in Aviation and Medicine: National, Organizational and Professional Influences* [Grundlagenwerk zum Crew Resource Management (CRM), das in der Selbstreflexion als Analogie herangezogen wird]. Ashgate.
- Marston, W. M. (1928). *Emotions of Normal People* [Grundlage des DISC-Verhaltensmodells, das Kommunikationstypen in vier Dimensionen einteilt: Dominant (rot), Initiativ (gelb), Stetig (grün), Gewissenhaft (blau)]. Kegan Paul, Trench, Trübner & Co.
- Mercedes-Benz Group AG. (2024a). *Geschäftsbericht 2024*. Verfügbar 16. Februar 2026 unter <https://group.mercedes-benz.com/investors/reports-news/annual-reports/2024/>
- Mercedes-Benz Group AG. (2024b). *MB.OS – Das Mercedes-Benz Operating System*. Verfügbar 16. Februar 2026 unter <https://group.mercedes-benz.com/innovation/digitalisation/connectivity/mb-os.html>
- Mercedes-Benz Group AG. (2024c). *Unternehmensstrategie*. Verfügbar 16. Februar 2026 unter <https://group.mercedes-benz.com/unternehmen/strategie/>
- Sinek, S. (2019). *The Infinite Game*. Portfolio/Penguin.

Glossar

- ADX** Azure Data Explorer 9, 11–13, 15, 16, 28
- API** Application Programming Interface 13, 18, 22
- CAN** Controller Area Network 1, 20–22, 25, 26
- CSV** Comma-Separated Values 18, 19
- DLT** Diagnostic Log and Trace 10, 11, 13, 18–20, 25, 28
- GUI** Graphical User Interface 13, 25
- HU** Head Unit 10, 22, 25
- Kusto** Abfragesprache (Kusto Query Language, KQL) für Azure Data Explorer, optimiert für die explorative Analyse großer Datenbestände 16, 23
- MariaDB** Relationale Open-Source-Datenbank, die in der DLT-Analyse-Pipeline als Zwischenspeicher zwischen CarLa-Filtern und Splunk dient 18, 19, 27
- MB.OS** Mercedes-Benz Operating System 9, 19
- Monaco** Diagnosewerkzeug für Fahrzeugsteuergeräte, das bei Mercedes-Benz zur Kommunikation mit Steuergeräten über Diagnoseprotokolle eingesetzt wird 21, 28
- OpenCV** Open Source Computer Vision Library – Bibliothek für Bildverarbeitung und Videoaufnahme, die im Projekt real-key-handly zur Videoaufnahme eingesetzt wurde 13

OTA Over-the-Air 9, 10, 25

Page Freeze Technik zur Unterbrechung von JavaScript-Ausführung in einer Webseite, um den aktuellen DOM-Zustand zu erfassen, bevor dynamische Frameworks (z. B. Angular.js) die Daten überschreiben 13

Preflight Check Aus der Luftfahrt übernommenes Konzept: eine systematische Prüfliste, die vor dem Start einer Test-Session alle relevanten Parameter verifiziert, um Konfigurationsfehler frühzeitig zu erkennen 25

Splunk Big-Data-Plattform zur Echtzeitsuche, -überwachung und -analyse großer Datenmengen. Bei Mercedes-Benz wird Splunk zur Visualisierung von Fahrzeugdiagnosedaten eingesetzt 9, 11, 15, 18–20, 28

TCU Telecommunication Unit 10, 22, 25

VAM Vehicle Assessment Module 12

VeDoc Vehicle Documentation – internes System zur Verwaltung von Fahrzeugdatenkarten und Konfigurationsinformationen bei Mercedes-Benz 21

VVR Virtual Vehicle Representation 15, 16, 21

wxPython Python-Bibliothek für native grafische Benutzeroberflächen, basierend auf dem wxWidgets-Framework 13

Erklärung zur wissenschaftlichen Arbeit

Ich versichere hiermit, die Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt, die wörtlich oder inhaltlich übernommenen Stellen als solche kenntlich gemacht und die Allgemeine Studien- und Prüfungsordnung für das Bachelor- und Masterstudium der Hochschule Reutlingen, die fachspezifische Studien- und Prüfungsordnung und die Regeln zur Sicherung der guten wissenschaftlichen Praxis der Hochschule Reutlingen beachtet zu haben.

Diese Arbeit oder Teile dieser Arbeit sind weder Bestandteil einer anderen Prüfungsleistung an dieser noch an einer anderen wissenschaftlichen Institution.



Reutlingen, 24. Februar 2026
Lauterbach, Martin